

ACTIVATION SPARSIFICATION IN TRANSFORMERS: THRESHOLD PLACEMENT, PRESSURE PLACEMENT, AND SPARSE EXECUTION

Anonymous authors

Paper under double-blind review

ABSTRACT

Activation sparsity can accelerate language-model inference when specialized kernels skip computation associated with zero activations. Existing methods obtain sparsity through different combinations of training-time pressure, thresholding, and placement, but their relative and combined effects have not been isolated under a common setup. As a result, it remains unclear which interventions drive the gains, when they are complementary or redundant, and how much model-wide computation they actually expose to sparsification. We study these interventions under matched pretraining conditions across Pythia models. To measure their computational potential, we define model-wide activation sparsity $\mathcal{S}_{\text{model}}$ as the fraction of forward matrix-multiplication products with a zero activation operand, together with an architecture-dependent sparsity ceiling $\mathcal{S}_{\text{model}}^{\text{max}}$. Pressure placement changes the quality–sparsity trade-off: in the 14M sweep, h-only pressure gives favorable moderate-threshold operating points, while broad seven-site pressure becomes useful for extreme sparsity. The broad-pressure recipes reach 27.48%, 40.60%, and 80.62% $\mathcal{S}_{\text{model}}$ at 14M, 70M, and 410M parameters, corresponding to 0.918, 0.822, and 0.924 of their respective sparsity ceilings. Selected trained recipes also outperform uniform post-hoc thresholding at high sparsity across all model sizes. For 35 measured 14M checkpoints on an RTX5090, structured projection work bypass is more strongly associated with projection-skipping gain ($R^2 = 0.941$) than scalar projection sparsity ($R^2 = 0.425$); larger logical sparsity need not imply lower optimized latency. These results show that activation sparsification is governed by the interaction between how zeros are induced, where they are placed, and how much of that zero structure can be exploited by sparse execution.

1 INTRODUCTION

Activation sparsity can improve language-model inference efficiency when specialized kernels skip computations associated with zero activations. Recent work has shown that sparsity can be exposed in pretrained models through post-hoc thresholding or induced during training, in both cases with measured inference speedups (Liu et al., 2024; Cetin et al., 2026). These results raise a practical design question: how to effectively sparsify a dense model in activations into an exploitable structure for optimized inference?

Existing approaches induce activation sparsity in different ways. TEAL applies post-hoc clipping to pretrained models, ProSparse uses ℓ_1 regularization on feed-forward (FFN) activations during training, and Q-Sparse applies top-k sparsification to both FFN and attention activations (Liu et al., 2024; Song et al., 2024; Wang et al., 2024). While each approach is effective in its own setting, it remains unclear how they compare under a common setup.

To address this gap, we pretrain Pythia-family models from scratch for controlled comparisons across model sizes (Biderman et al., 2023) to study the effects of (i) activation ℓ_1 pressure (P) and (ii) *thresholding* (T). While pressure adds a gradient component to induce smaller activation magnitudes, thresholding maps small values into exact zeros. Each one of them can be placed in different activation sites, giving a combinatorial design set of sparse interventions. For each com-

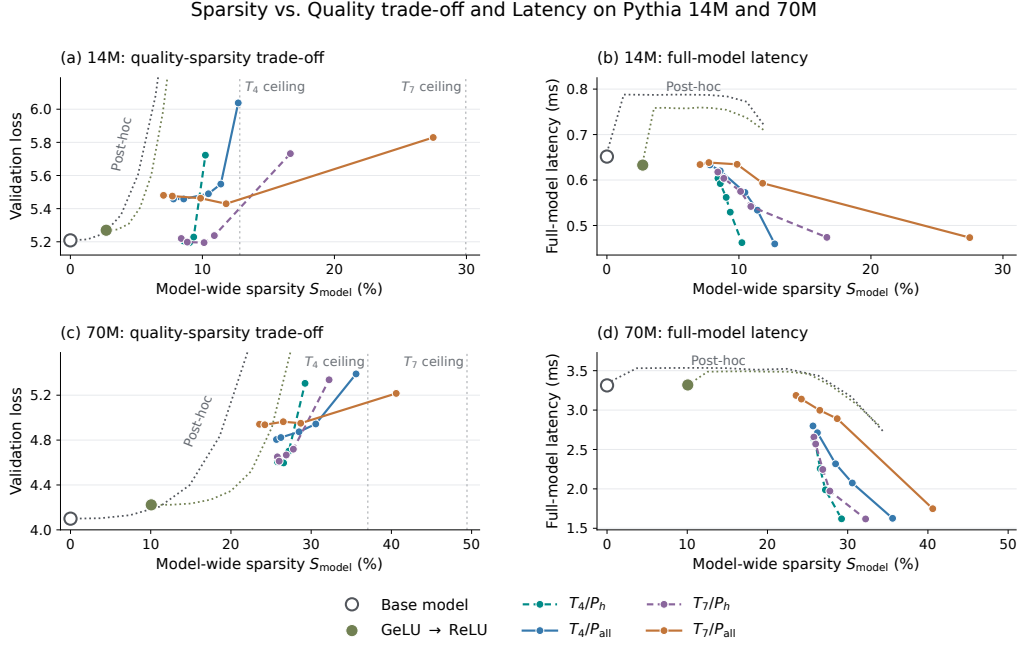


Figure 1: **Quality and execution depend on threshold and pressure placement.** Rows show Pythia-14M (a,b) and 70M (c,d); columns show final validation loss and full-model latency versus S_{model} . Each row contains 22 checkpoints: the Base Model (T_0/P_0), GeLU $\rightarrow$ ReLU at the hidden FFN activation (h) (T_1/P_0), and T_4/T_7 thresholding at four/seven sites with pressure applied either at h (P_h , dashed) or at all thresholded sites (P_{all} , solid). Figure 2 and Table 1 describe the intervention sites. The T/P curves connect separate runs with parameters controlling the degree of sparsity, while dotted post-hoc paths clip the fixed controls for $p = 0, .1, \dots, .9$. Vertical dashed lines mark the analytic sparsity ceilings.

bination of P and T , we derive an architecture-dependent maximum achievable sparsity ($S_{\text{model}}^{\text{max}}$), and measure model-wide attained activation sparsity (S_{model}) and the final inference latency under a specialized sparse kernel. Figure 1 presents sparsity vs. quality trade-offs and inference latency for 14M and 70M Pythia models, across P/T combinations and post-hoc clipping of baselines.

Across 36 14M-parameter and 22 70M-parameter training and evaluation runs, we identify several useful insights. (1) First, the T/P train-time interventions extend the attainable sparsity range relative to post-hoc clipping, approaching the architectural ceiling for the more aggressive recipes, but at a cost in quality. (2) Second, sparsity and speedup increase together within recipes; however, although they are correlated, a larger S_{model} does not directly translate into a larger speedup. The main requirement for lower latency is the location and structure of zeros and whether the kernel can exploit them, with most gains coming from skipped computation in FFN-hidden layers (h) and the attention output projection (z). (3) Third, for exploitable sparse structures, increasing model scale increases the opportunity for latency reductions. (4) Fourth, local FFN-hidden (h) pressure provides a more favorable balance between quality, sparsity and latency than global pressure when combined with broader thresholding. (5) Finally, local pressure can induce global changes in sparsity, showing that interventions targeted at individual activation sites can affect the model-wide sparsity structure. Taken together, these results indicate that activation sparsification should not be treated as an isolated training objective: training interventions and deployment kernels must be co-designed so that the induced sparsity is both high and structurally exploitable at inference time.

2 RELATED WORK

2.1 SPARSIFICATION INTERVENTIONS

Post-hoc thresholding removes small activations at inference-time of a pre-trained model. While CATS thresholds small-magnitude activations within FFNs (Lee et al., 2024), TEAL extends it to attention (Liu et al., 2024). These studies motivate training-time interventions to adapt activation distributions to sparse computation and improve the trade-off between quality and sparsity.

ReLU Strikes Back studies the effect of activations functions in language models (Mirzadeh et al., 2023), showing clear sparsity gains on the use of ReLUs. ProSparse combines progressive ℓ_1 pressure and thresholding at FFNs during continued training (Song et al., 2024). It does not, however, extend its interventions to attention and restrict the analysis to local FFN sparsity. Cetin et al. (2026) extends ProSparse setup to pre-training and propose specialized kernels. These studies show that training can reshape the quality–sparsity trade-off, but they’re restricted to a single local intervention at FFN sites.

Q-Sparse extend sparsification beyond FFNs. It applies top- k selection to both attention and FFN projection inputs (Wang et al., 2024), but with no ℓ_1 pressure. Its top- k operation introduces, however, additional overhead due to the sorting operation. Spark Transformer tries to address it with statistical top- k (You et al., 2025), but we take a simpler approach and use a constant thresholding non-linearity. We argue that when the intervention is introduced during training, the network can adapt itself. Additionally, we evaluate the complementary effect of regularization pressure on both attention and FFNs.

2.2 INFERENCE SPEEDUP

Turning activation sparsity into speedup requires a specialized kernel adapted to the model architecture. TEAL accelerates batch-one autoregressive decoding, reporting up to $1.8\times$ speedup at 50% sparsity (Liu et al., 2024). Cetin et al. (2026) address batched matrix multiplication with their TwELL kernel, and report $1.3\times$ speedup for batch inference. Spark likewise couples its sparsity mechanism with an implementation that reports decoding gains (You et al., 2025). In each setting, skipped computation and memory access must compesate the additional overhead introuced by the sparse operations. This development has been increasingly accelerated by the use of coding agents, through iterative kernel development with execution and profiling feedback (Ouyang et al., 2025; Luo et al., 2026). We take a related approach and use GPT-6 Astra do develop a specialized kernel based on our training interventions (OpenAI, 2026).

3 METHODOLOGY

3.1 SPARSIFICATION INTERVENTIONS

A sparsification intervention specifies where activations are thresholded (thresholding sites $\mathcal{N}$) and where activation pressure is applied (pressure sites $\mathcal{P}$). The sets $\mathcal{N}$ and $\mathcal{P}$ need not coincide, and the thresholding and pressure functions may take different forms. Together, these choices define a combinatorial space of training interventions. We restrict this space to keep the computational cost manageable while addressing the central research questions.

For thresholding, we avoid the overhead of exact top- k selection (Wang et al., 2024; You et al., 2025) and instead use one-sided and symmetric fixed-threshold operators $G_{+,\kappa}(x) = x\mathbf{1}\{x \geq \kappa\}$ and $G_{\pm,\kappa}(x) = x\mathbf{1}\{|x| \geq \kappa\}$, with $\kappa \geq 0$. The constant κ controls the strength of the intervention and is held fixed within each training condition. At $\kappa = 0$, $G_{+,\kappa}$ reduces to ReLU, whereas $G_{\pm,\kappa}$ reduces to the identity. Appendix A specifies the boundary and derivative conventions.

For activation pressure, ℓ_1 averages, across targeted sites and layers, the mean absolute activation at each site after any thresholding. Training then minimizes $\mathcal{L}_{\text{task}} + \lambda\ell_1$, where λ controls the sparsity–quality trade-off (Song et al., 2024; Cetin et al., 2026). To limit conflict between the ℓ_1 pressure and the task update while reducing the need to tune an additional hyperparameter, we introduce orthogonal ℓ_1 pressure (OL1), inspired by gradient-conflict projection methods (Yu et al., 2020; Hsieh et al., 2024). OL1 chooses the step that maximizes alignment with the preconditioned

pressure direction, within a unit norm budget relative to the adaptive Adam direction and without opposing it. At a fixed optimizer state, once the norm budget is active, further increasing λ leaves the resulting pressure step unchanged. To keep the multisite intervention study tractable, we use a common $\lambda = 1$ and budget $b = 1$ and directly measure whether the budget becomes active during training. Section 4.2 shows that budget saturation depends strongly on the target set rather than occurring universally. Appendix A.2 gives the full formulation, a brief comparison between L1 and OL1, and shows that budget saturation depends strongly on the target set rather than occurring universally.

Figure 2 locates the intervention sites within the Pythia architecture. The variables a and m denote the normalized inputs to the attention and FFN branches, respectively; h is the FFN hidden activation; z is the concatenated attention context before the output projection; and q , k , and v are the query, key, and value operands, with q and k taken after RoPE. Attention probabilities are denoted by p . We apply $G_{+, \kappa}$ at selected sites in $\{a, m, h, z\}$ and $G_{\pm, \kappa}$ at selected sites in $\{q, k, v\}$. When h is targeted, the thresholding nonlinearity replaces GELU.

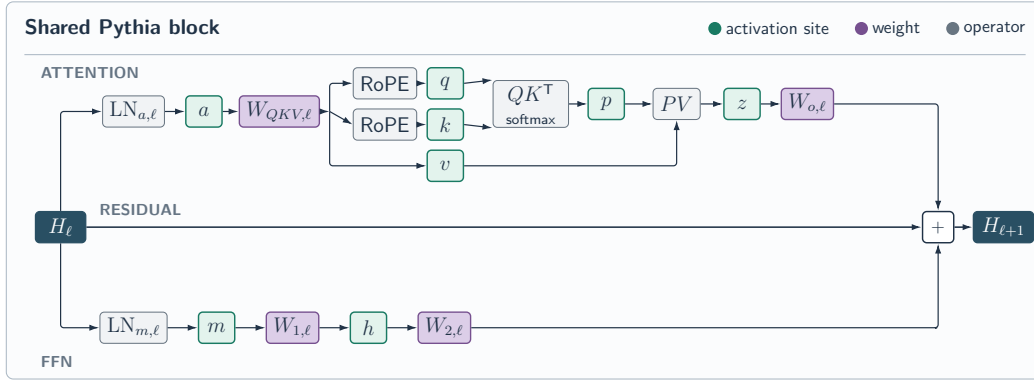


Figure 2: **Intervention sites in a shared Pythia block.** The attention and FFN branches use separate LayerNorms and join the residual path. Green nodes denote activation sites, and purple nodes denote learned weights. Table 1 specifies the thresholding and pressure assignments.

3.2 SPARSITY MEASURE AND ARCHITECTURAL REACH

Activation sparsity is the fraction of zero activations at a site. We define *model-wide sparsity* as $\mathcal{S}_{\text{model}} = N_0/N_{\text{model}}$, where N_0 counts scalar multiplications with a zero activation operand in Pythia-block matrix products, and N_{model} counts all scalar multiplications in those blocks and the final output projection. We aggregate counts over validation data, excluding masked attention pairs (Appendix B).

To quantify intervention potential a priori, we define the *model-wide sparsity ceiling* as $\mathcal{S}_{\text{model}}^{\text{max}} = N_{\text{reach}}(\mathcal{N})/N_{\text{model}}$, where $N_{\text{reach}}(\mathcal{N})$ counts scalar multiplications guaranteed a zero activation operand if all selected thresholding sites are 100% sparse. Thus, $\mathcal{S}_{\text{model}}^{\text{max}}$ is the maximum sparsity directly attributable to $\mathcal{N}$. It depends on architecture, selected sites, and sequence length. Table 1 shows that larger models can have higher ceilings at fixed sequence length, because FFN layers are more representative on the model total computations. Natural zeros outside the intervention reach can make $\mathcal{S}_{\text{model}} > \mathcal{S}_{\text{model}}^{\text{max}}$. Appendix B.2 gives the precise accounting.

4 EXPERIMENTAL STUDY

We pretrain Pythia-14M, 70M, and 410M from random initialization (Biderman et al., 2023) on MiniPile (Kaddour, 2023), using 1.493 billion input tokens per model and training for one full-pass. We evaluate validation loss and $\mathcal{S}_{\text{model}}$ over all complete validation sequences. Table 1 defines the thresholding and pressure recipes and their sparsity ceilings across model sizes. It shows how the attainable sparsity ceiling increases with model size, under the same intervention recipe and fixed sequence length.

Experiments across model sizes. The 14M runs provide the most extensive controlled ablation, used to identify threshold and pressure regimes. The 70M runs test whether these regimes persist in a larger model. We also attempted to extend this analysis to 410M, but under the same token budget and number of optimization steps, this leaves the model substantially less trained. The baseline run with no intervention had a higher final loss and larger late-training gradient norm, which is evidence of a different training regime. We therefore treat the 410M result as a limited, partial scale-up, and focus the detailed intervention analysis on the 14M and 70M runs (Appendix C.4).

Within each model size and intervention, runs share initialization, data order and training budget. Appendix C.4 gives the full protocol. For comparison, we apply TEAL-style post-hoc magnitude thresholding (Liu et al., 2024) at a, m, h, z , with percentile targets $p = 0, 0.1, \dots, 0.9$ and site-layer thresholds calibrated separately on ten training blocks.

Table 1: Thresholding, pressure and sparsity ceilings. T_1 replaces GeLU with ReLU at h . Pressure uses OL1: $P_1 = P_h$ targets h , and P_{all} matches the thresholding sites ($\mathcal{P} = \mathcal{N}$). For T_4/T_7 , threshold strength is swept over $\kappa \in \{0, 0.01, 0.05, 0.1, 0.5\}$, with OL1 pressure weight $\lambda = 1$ and task-relative norm budget $b = 1$ fixed. Only the 14M T_1/P_1 ablation sweeps $\lambda \in \{0.05, 0.1, 0.5, 1\}$ at $b = 1$. Ceilings are $100 \cdot \mathcal{S}_{\text{model}}^{\max}$ at 2,048 tokens; executed recipes by model size are listed in Appendix C.4.

Recipe	Threshold sites $\mathcal{N}$	Pressure sites $\mathcal{P}$	Ceiling (%)		
			14M	70M	410M
T_0/P_0 (Base Model)	$\emptyset$	$\emptyset$	0.00	0.00	0.00
T_1/P_0	h	$\emptyset$	4.28	12.35	24.93
T_1/P_1	h	h	4.28	12.35	24.93
T_4/P_0	a, m, h, z	$\emptyset$	12.83	37.06	74.78
T_4/P_h	a, m, h, z	h	12.83	37.06	74.78
T_4/P_{all}	a, m, h, z	$\mathcal{N}$	12.83	37.06	74.78
T_7/P_0	a, m, h, z, q, k, v	$\emptyset$	29.95	49.42	87.25
T_7/P_h	a, m, h, z, q, k, v	h	29.95	49.42	87.25
T_7/P_{all}	a, m, h, z, q, k, v	$\mathcal{N}$	29.95	49.42	87.25

Quality-sparsity trade-offs on Pythia 14M and 70M

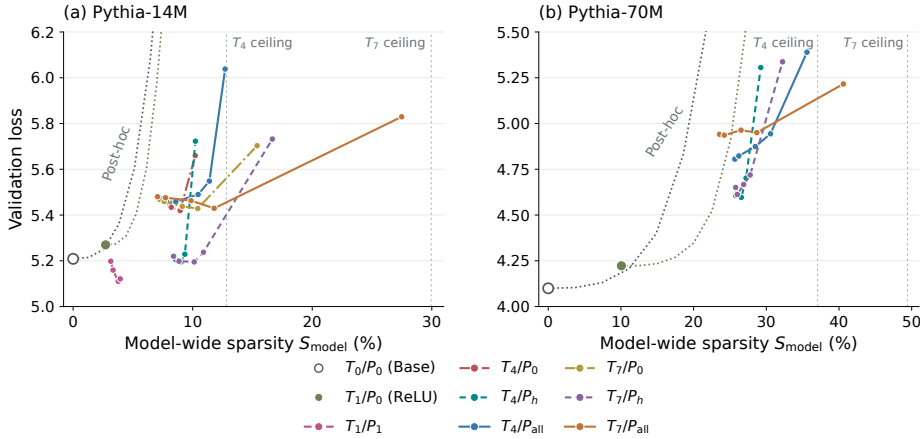


Figure 3: Quality-sparsity trade-offs at 14M and 70M. 36/22 trained conditions, with loss scales fitted separately to each size. Colors distinguish executed recipes; lines connect separate threshold settings (κ), or pressure weights (λ) for T_1/P_1 . Dotted paths apply post-hoc clipping at a, m, h, z to the fixed base and ReLU controls. Vertical guides mark the T_4 and T_7 analytic sparsity ceilings.

The interventions extend the attainable sparsity range, sometimes close to the architectural ceiling, but at a quality cost. Figure 3 summarizes 58 trained conditions at 14M and 70M. At $\kappa = 0.5$, T_7/P_{all} reaches 27.48% and 40.60% model-wide sparsity: 91.8% and 82.2% of the respective ceilings. Validation loss increases by 0.62 and 1.12 relative to the same-size base model. At intermediate $\kappa \in 0.01, 0.05$, local pressure at h present a comparable sparsity at lower loss, compared to global pressure P_{all} . At subsection 4.2 we do a paired analysis on the effect of global vs. local pressure.

4.1 FROM SPARSITY TO SPEEDUPS

Agentic kernel development. To test whether learned sparsity reduces inference latency, we used Codex with GPT-6 Astra (OpenAI, 2026), to develop specialized GPU kernels under human guidance. We started from the kernels released by Cetin et al. (2026), although they required adaptation for Pythia architecture. The agent proposed, implemented and tested the kernels speed and accuracy. During the development phase, we sampled a subset from the training set and tested all kernel changes on them, comparing the model’s output scores and loss with and without the custom GPU implementation. We rejected kernels that provided a different model output response. Once we found a kernel that provided speedup for a single sparse model with the same response, we tested it for each training recipe. For each model, we measured inference time and output correctness over the validation set. Its speedup was recorded and included only if these checks passed. We then repeated this evaluation at 70M after adapting the kernels to its larger matrix dimensions (Appendix D.4). Although we are not specialized in GPU-kernel design, this workflow made direct execution tests practical. This case illustrates how coding agents can support systems development.

Sparsity and speedup increase together within recipes. For the implemented kernel, Figure 4 shows an approximately linear increase within each trained threshold/pressure family over the tested grid, although with different slopes across families. At 14M and $\kappa = 0.5$, for example, T_7/P_{all} is much sparser than T_4/P_{all} (27.48% versus 12.71%), yet slightly slower ($1.38\times$ versus $1.42\times$ base-model speedup). Although they correlate, a larger $\mathcal{S}_{\text{model}}$ does not directly translate into a larger speedup. The main requirement is the place and structure of zeros and wheater the kernel can exploit them. We dive in the reasons for it under the implemented kernel and training recipes on Section 4.3.

For exploitable structure, scale increases the opportunity. Figure 4 shows that the attainable full-model speedup increases with model size. Larger models combine larger matrix operations with higher sparsity ceilings, making sparse computation more effective at reducing end-to-end latency. Accordingly, the best observed speedup rises from $1.42\times$ at 14M to $2.08\times$ at 70M.

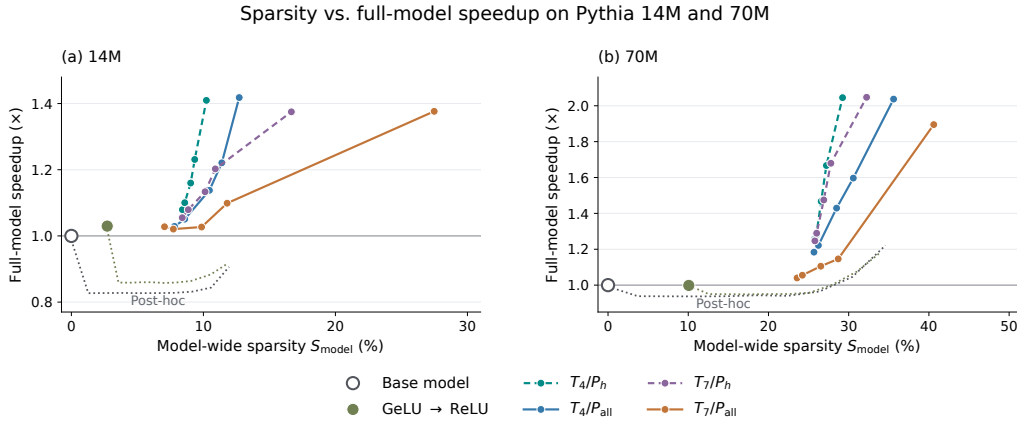


Figure 4: From model-wide sparsity to measured speedup. Pythia-14M (a) and 70M (b), with 22 trained checkpoints and 20 post-hoc clipping settings per size. Speedup divides the same-size base-model latency by each setting’s latency, using the final specialized implementation for both terms. Timings use RTX5090, BF16, batch one, 2,048 tokens and the full logit head. Sparsity uses full-validation pooled logical counts.

Post-hoc clipping does not yield comparable gains in this implementation. Every post-hoc clipping point remains below $1\times$ at 14M, and the first degradation point show the additional overhead cost. At 70M, aggressive clipping can exceed $1\times$, reaching $1.22\times$, but with substantial loss degradation (Appendix D.3). Here clipping uses a separate masking operations around the kernel, adding additional overhead because it has to iterate over activations on every forward pass. The resulting zeros still have to form groups that the kernel skipping rules can use. The overhead can outweigh the avoided products. However, we did not optimize a fused clipping implementation, so this result does not establish a general limitation of post-hoc sparsification, but **points into a requirement of pairing the sparse intervention with specialized implementation.**

4.2 PAIRED EFFECTS OF GLOBAL VS. LOCAL PRESSURE

One question raised during sparse intervention design is wheater activation pressure should be applied over the entire network, or are there local sites more suceptible to sparsification? To answer this, we compare thresholding accross multiple sites (T_4 and T_7) with global and local FFN hidden pressure (P_{all} and P_h) at Figure 5.

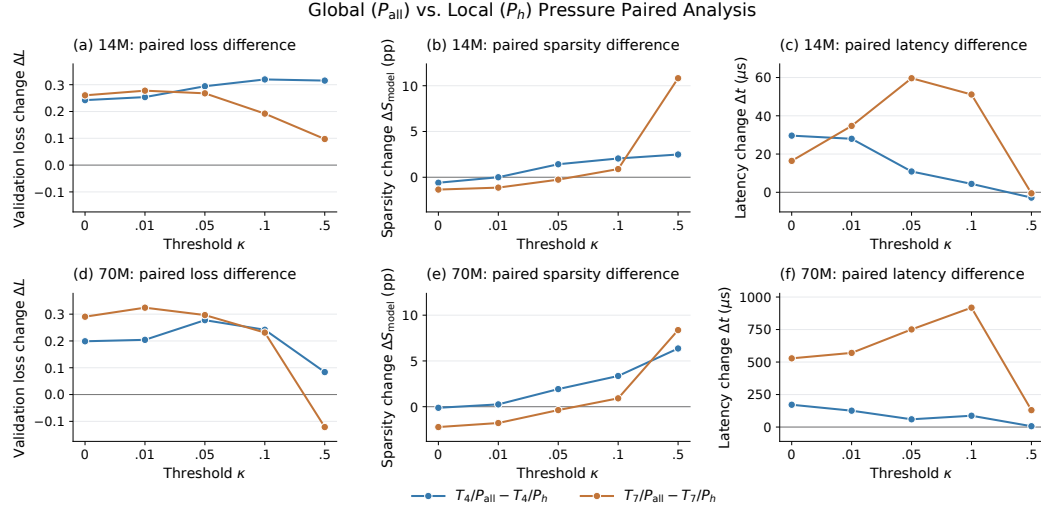


Figure 5: **Paired effects of broadening pressure from P_h to P_{all} .** Every point is P_{all} minus P_h at matched model size, threshold sites and κ , with $\lambda = b = 1$. Rows show 14M/70M; columns show loss change, sparsity change (pp), and full-model latency change (μs). Positive loss or latency differences favor P_h ; positive sparsity differences favor P_{all} . Latencies subtract checkpoint geometric means: RTX5090, BF16, batch one, 2,048 tokens, full logit head and 1,344 timings per checkpoint.

Local h-only pressure gives a more favorable quality–sparsity trade-off at moderate thresholds in both model sizes. For $\kappa \leq 0.1$, P_h lowers loss relative to P_{all} in all 16 matched comparisons at 14M and 70M (Figure 5). The loss reduction is 0.19–0.32, with sparsity differing by at most 3.36 percentage points.

The additional sparsity from global pressure does not imply in lower latency. H-only pressure has lower measured latency in 18 of the 20 pairs, including all ten 70M pairs. Thus, the extra sparsity from broader pressure does not consistently translate into lower latency. Detailed 14M pressure-budget diagnostics are reported in Appendix D.6.

Local pressure can produce nonlocal sparsity changes. Figure 6 present the sparsity by site and layer accross T_4 and T_7 interventions, for all pressures at 14M. At $\kappa = 0.05$, P_h increases sparsity at both h and z relative to P_0 , although z receives no direct pressure. Under T_7 , P_h also produces higher sparsity at h and z than P_{all} , despite the latter targeting both sites. One possible explanation is that P_{all} caps the OL1 norm more frequently (Appendix D.6) and therefore the local pressure at h is smaller at P_{all} . The takeaway is that a local site pressure may spillover and affect the distributions of other sites.

(h, z) are more susceptible to sparsification than attention sites (q,k,v). At higher threshold ($\kappa = 0.5$) (h,z) sites become approximately 100% sparse, even if no pressure is applied, while (q,k,v) sites are in the [14, 36] sparsity range. This implies the distribution of attention sites may have a different range and require different threshold values to avoid hurting the model quality too much.

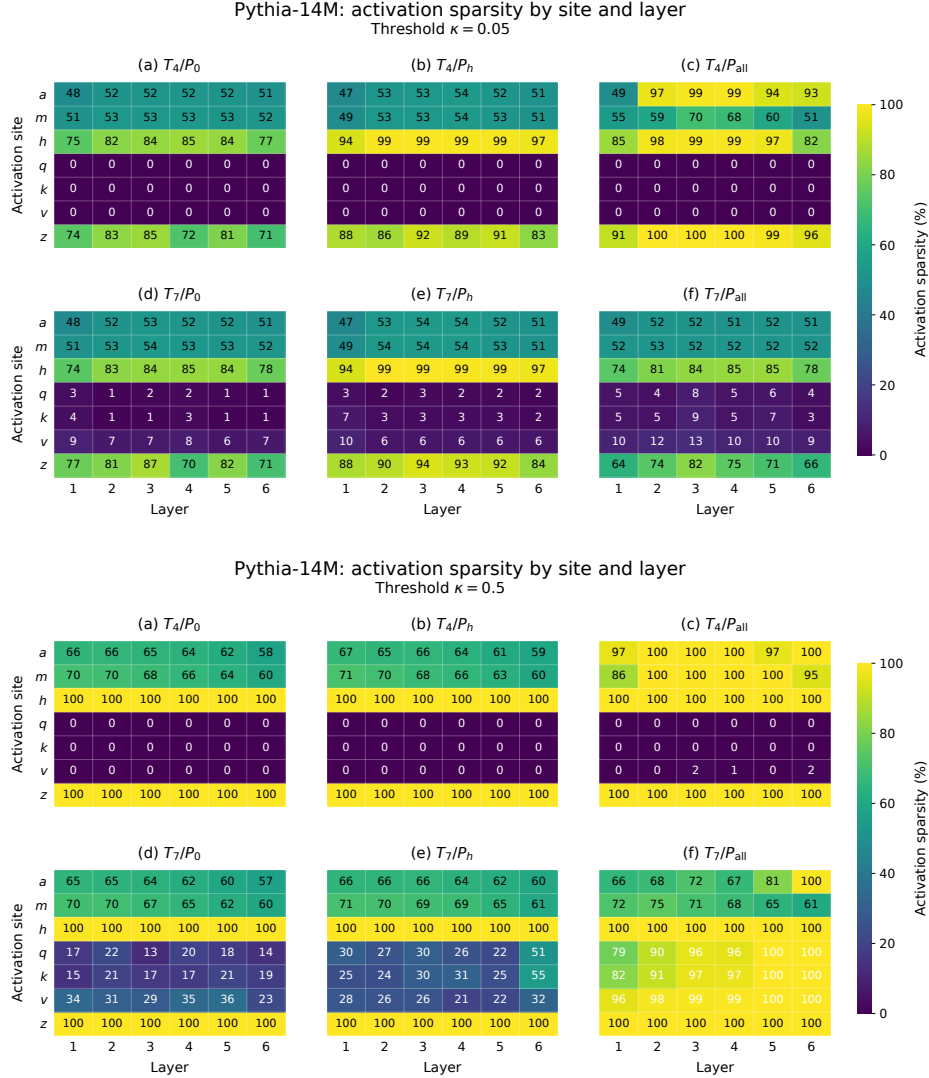


Figure 6: Activation sparsity by site and layer in Pythia-14M. The six threshold/pressure recipes at $\kappa = 0.05$ (top) and 0.5 (bottom). Each cell gives the percentage of zero activations over all complete validation sequences at one site and layer. Q/K are post-RoPE; all panels share a 0–100% color scale. Values are rounded to whole percentages, so 0 or 100 does not establish a completely dense or all-zero tensor.

4.3 WHEN MODEL-WIDE SPARSITY TRANSLATES TO SPEEDUP

Sparsity reduces latency when avoided work exceeds the overhead cost of detecting and skipping it. At h, z , the sparse kernel checks 8×16 activation tiles before loading their associated weights, so empty tiles avoid both weight load from memory and the matrix multiplications. This strategy, however, doesn’t extend to other sites. Applying it at a and m increased full-model latency by 17.2% and 27.4% on 14M T_7/P_{all} at $\kappa = 0.5$, with unchanged outputs. Only 11.12% of a ’s 8×16 tiles

were empty, and none at m , so most groups still required matrix multiplication and the additional overhead was not justified.

Attention follows FlashAttention’s tiled sequence (Dao et al., 2022): load Q, K , compute scores, apply masking and softmax, then multiply probabilities by V . Our implementation add additional overhead after the checks, so skipping an empty tile’s multiplication cannot recover the latency from reading from memory. Softmax still runs and turns zero scores into nonzero probabilities, so Q/K sparsity does not automatically propagate to PV . Zeros in V remain usable, and sparsifying the probabilities is a further untested opportunity.

We isolate each site’s effect at 14M by disabling it during inference time. Table 2 reports the *bypass* (i.e. percentage of skipped matrix instructions) and saved time per-site sparse operation for the sparser 14M recipe T_7/P_{all} , $\kappa = 0.5$.

Table 2: **Conditional latency savings: 14M T_7/P_{all} , $\kappa = 0.5$.** Times are per 2,048-token sequence on RTX 5090 (BF16, batch one). Saved time is full-model latency with one skipping path off minus latency with all paths on; positive values indicate a benefit. Brackets give [minimum, maximum] savings across all off/on pairs of three independent timing runs per mode, not confidence intervals. Each run reports the geometric mean over 64 sequences and seven passes. Effects are not additive.

Operation	Bypass (%)	Saved time (μs)	Process span (μs)
a : QKV projection	8.4	-1.6	$[-6.0, +3.8]$
m : FFN-up	0.0	-2.4	$[-5.1, +2.7]$
h : FFN-down	94.5	+150.1	$[+145.4, +154.4]$
z : attention-output projection	99.6	+29.0	$[+23.7, +35.7]$
q, k : QK scores	58.0	-2.9	$[-7.3, +1.1]$
v : PV product	67.2	-6.2	$[-9.0, -2.2]$

It makes clear that savings come from h (150 μs) and z (29 μs). Individual a, m, QK effects remain unresolved against process variation, whereas PV skipping adds latency. Enabling QK/PV skipping together raises full-model latency from 0.514 to 0.521 ms despite substantial bypass. Thus, for this checkpoint and implementation, attention’s arithmetic savings do not offset its overhead, while h/z benefit from avoiding weight reads as well as computation.

5 DISCUSSION AND CONCLUSION

Our results suggest that sparsification should be understood as a joint training-and-deployment problem rather than as a property of activation distributions alone. First, **local pressure does not imply a local response**: applying pressure only at the FFN hidden site can alter sparsity at other sites, including z , showing that the network can reorganize its internal representations in response to a localized training intervention. More broadly, sparsity is useful only insofar as the resulting zero structure can be exploited by the inference system. S_{model} is therefore best viewed as a training-time proxy for computational opportunity, not as a direct predictor of latency: in our measurements, models with greater logical sparsity can be slower, while instruction-level projection bypass is substantially more predictive of projection-skipping gains than scalar sparsity. This connects training and deployment directly. Specialized kernels determine which activation patterns are valuable, while training interventions can encourage the model to acquire those patterns. Transformers appear sufficiently flexible to adapt their internal representations to architectural and optimization constraints, suggesting that larger gains may come from **designing training objectives around inference requirements**, rather than first maximizing a generic sparsity metric and only afterward attempting to accelerate it. The same principle may extend beyond activation sparsity: training could be used to shape representations for properties required by weight sparsity, quantization, batched execution, or other hardware- and workload-specific constraints. In this view, deployment requirements are not merely constraints imposed after training, but potential signals for deciding what properties the network should learn.

Several limitations define the scope of these conclusions and point to concrete directions for further work. Our scaling evidence is limited: although the 410M experiments demonstrate that high model-wide sparsity remains attainable, the fixed token budget leaves 410M in a substantially differ-

ent optimization regime, so it should be interpreted as a stress test rather than a clean scaling experiment. Our runtime study is also restricted to batch-one, full-sequence inference on an RTX5090; we do not study how sparsity interacts with batched inference, cached decoding, larger models, or other hardware, all of which can change the relative importance of compute, memory movement, and sparsity-detection overhead. In addition, the full intervention space is combinatorial, and under the available compute budget we evaluate only a selected set of threshold and pressure placements rather than an exhaustive factorial study. The nonlocal response visible in Figure 6 suggests one particularly promising direction: interventions concentrated at h and z may preserve much of the projection-side execution benefit while avoiding pressure or thresholding at sites whose sparsity is expensive in quality or unprofitable for the current kernel. Related extensions include learning or tuning different thresholds κ at different sites and directly sparsifying attention probabilities after softmax, which could expose work in PV without relying on Q/K zeros to propagate through softmax. More generally, rather than asking where the model can be made sparse, a productive systems-guided approach is to decompose full-model inference into individual operations, identify where latency and removable work are concentrated, determine what structured representation would allow those operations to be skipped efficiently, and then design training-time interventions that encourage precisely that structure.

REFERENCES

- Stella Biderman, Hailey Schoelkopf, Quentin Gregory Anthony, Herbie Bradley, Kyle O’Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, Aviya Skowron, Lintang Sutawika, and Oskar van der Wal. Pythia: A Suite for Analyzing Large Language Models Across Training and Scaling. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pp. 2397–2430. PMLR, 2023. URL <https://proceedings.mlr.press/v202/biderman23a.html>.
- Edoardo Cetin, Stefano Peluchetti, Emilio Castillo, Akira Naruse, Mana Murakami, and Llion Jones. Sparser, Faster, Lighter Transformer Language Models. *arXiv preprint arXiv:2603.23198*, 2026. URL <https://arxiv.org/abs/2603.23198>.
- Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. *arXiv preprint arXiv:2205.14135*, 2022. URL <https://arxiv.org/abs/2205.14135>.
- Yu-Guan Hsieh, James Thornton, Eugene Ndiaye, Michal Klein, Marco Cuturi, and Pierre Ablin. Careful with that Scalpel: Improving Gradient Surgery with an EMA. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 19085–19100. PMLR, 2024. URL <https://proceedings.mlr.press/v235/hsieh24a.html>.
- Jean Kaddour. The MiniPile Challenge for Data-Efficient Language Models. *arXiv preprint arXiv:2304.08442*, 2023. URL <https://arxiv.org/abs/2304.08442>.
- Donghyun Lee, Je-Yong Lee, Genghan Zhang, Mo Tiwari, and Azalia Mirhoseini. CATS: Contextually-Aware Thresholding for Sparsity in Large Language Models. In *Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=v3w2a7EInO>.
- James Liu, Pragaash Ponnusamy, Tianle Cai, Han Guo, Yoon Kim, and Ben Athiwaratkun. Training-Free Activation Sparsity in Large Language Models. *arXiv preprint arXiv:2408.14690*, 2024. URL <https://arxiv.org/abs/2408.14690>.
- Ilya Loshchilov and Frank Hutter. Decoupled Weight Decay Regularization. In *International Conference on Learning Representations*, 2019. URL <https://arxiv.org/abs/1711.05101>.
- Mao Luo, Hongbin Li, Feng Lin, Hanling Yi, and Zhe Huang. Agentic Kernel Optimization: Generating State-of-the-Art GPU Kernels Without Hand-Written CUDA. *arXiv preprint arXiv:2608.14560*, 2026. URL <https://arxiv.org/abs/2608.14560>.

- Iman Mirzadeh, Keivan Alizadeh, Sachin Mehta, Carlo C. Del Mundo, Oncel Tuzel, Golnoosh Samei, Mohammad Rastegari, and Mehrdad Farajtabar. ReLU Strikes Back: Exploiting Activation Sparsity in Large Language Models. *arXiv preprint arXiv:2310.04564*, 2023. URL <https://arxiv.org/abs/2310.04564>.
- OpenAI. GPT-6 Astra: A New Generation of Intelligence. OpenAI, September 2026. URL <https://openai.com/index/gpt-6-astra/>. Accessed September 19, 2026.
- Anne Ouyang, Simon Guo, Simran Arora, Alex L. Zhang, William Hu, Christopher Ré, and Azalia Mirhoseini. KernelBench: Can LLMs Write Efficient GPU Kernels? In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 47356–47415. PMLR, 2025. URL <https://proceedings.mlr.press/v267/ouyang25a.html>.
- Chenyang Song, Xu Han, Zhengyan Zhang, Shengding Hu, Xiyu Shi, Kuai Li, Chen Chen, Zhiyuan Liu, Guangli Li, Tao Yang, and Maosong Sun. ProSparse: Introducing and Enhancing Intrinsic Activation Sparsity within Large Language Models. *arXiv preprint arXiv:2402.13516*, 2024. URL <https://arxiv.org/abs/2402.13516>.
- Hongyu Wang, Shuming Ma, Ruiping Wang, and Furu Wei. Q-Sparse: All Large Language Models can be Fully Sparsely-Activated. *arXiv preprint arXiv:2407.10969*, 2024. URL <https://arxiv.org/abs/2407.10969>.
- Chong You, Kan Wu, Zhipeng Jia, Lin Chen, Srinadh Bhojanapalli, Jiaxian Guo, Utku Evci, Jan Wassenberg, Praneeth Netrapalli, Jeremiah J. Willcock, Suvinay Subramanian, Felix Chern, Alek Andreev, Shreya Pathak, Felix Yu, Prateek Jain, David E. Culler, Henry M. Levy, and Sanjiv Kumar. Spark Transformer: Reactivating Sparsity in FFN and Attention. *arXiv preprint arXiv:2506.06644*, 2025. URL <https://arxiv.org/abs/2506.06644>.
- Tianhe Yu, Saurabh Kumar, Abhishek Gupta, Sergey Levine, Karol Hausman, and Chelsea Finn. Gradient Surgery for Multi-Task Learning. In *Advances in Neural Information Processing Systems*, 2020. URL <https://arxiv.org/abs/2001.06782>.

A INTERVENTION DEFINITIONS

A.1 THRESHOLDING NONLINEARITIES

For finite $\kappa \geq 0$, both thresholding nonlinearities retain equality and leave retained values unchanged. The comparison mask is detached: the derivative with respect to the input is one for retained entries and zero for rejected entries. Consequently, $G_{+,0}$ and ReLU have identical forward values but differ at zero, where their derivatives are one and zero, respectively. $G_{\pm,0}$ is exactly the identity in both values and gradients. These nonlinearities use neither a straight-through estimator nor a top- k selection rule. Unselected sites retain their original transformations.

A.2 ACTIVATION PRESSURE AND OL1

For the J targeted site-layer tensors A_j , containing n_j entries each, the pressure objective is $\mathcal{L}_1 = J^{-1} \sum_{j=1}^J n_j^{-1} \sum_{i=1}^{n_j} |A_{j,i}|$. The tensors contain the executed site activations, after the non-linearity where one is applied. Absolute values and means are evaluated in float32. Each tensor receives equal weight regardless of its size; adding pressure sites changes both the objective and the weight of existing targets. Task and pressure losses are averaged over the same accumulation microbatches. For naive L1, the accumulated gradient of $\mathcal{L}_{\text{task}} + \lambda \mathcal{L}_1$ is globally norm-clipped before AdamW (Loshchilov & Hutter, 2019).

For OL1, let g_t and g_1 be the accumulated task and unweighted pressure gradients. We first clip g_t by its global norm and perform AdamW using only that clipped task gradient. Let $\hat{m}_t$ and $\hat{v}_t$ be its newly updated, bias-corrected moments. On parameters having both gradients and initialized optimizer state, define the adaptive task and pressure vectors $u = \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon_A)$ and $w = g_1 / (\sqrt{\hat{v}_t} + \epsilon_A)$. The pressure gradient is unclipped and has no first-moment buffer. Inner products and norms below pool all these eligible parameters. With $c = \langle u, w \rangle$, $q = \|u\|_2^2$, and numerical stabilizer $\epsilon > 0$, the conflict-conditioned projection is

$$\tilde{w} = \begin{cases} w - \frac{c}{q + \epsilon} u, & c < 0 \text{ and } q > \epsilon, \\ w, & \text{otherwise.} \end{cases} \quad (1)$$

For diagnostics, we define the task-relative opposing component

$$\rho_{\text{opp}} = \frac{\max(0, -c)}{q + \epsilon}.$$

When projection is active, this equals the norm of the component removed from w , divided by $\|u\|_2$; it is zero when $c \geq 0$. All reported steps satisfy the guard $q > \epsilon$. It is therefore not the fraction of the pressure norm removed.

For pressure weight $\lambda \geq 0$ and required budget $b > 0$, define the norm ratio $r = \lambda \|\tilde{w}\|_2 / (\|u\|_2 + \epsilon)$. Set $s = \min\{1, b/(r + \epsilon)\}$ when $r > 0$, or $s = 1$ otherwise. After AdamW, each eligible parameter group p , with learning rate α_p , receives $\theta_p \leftarrow \theta_p - \alpha_p \lambda s \tilde{w}_p$. The experiments' clipping thresholds, b , and λ are condition parameters. We report the norm budget as active when the implemented scale satisfies $s < 1$.

Ignoring numerical stabilizers, the correction norm before the learning rate is $\min\{\lambda \|\tilde{w}\|_2, b\|u\|_2\}$. For a fixed optimizer state, once the cap binds, further increases in λ leave this ideal correction unchanged.

This procedure is related to gradient-conflict projection (Yu et al., 2020) and the use of smoothed primary directions (Hsieh et al., 2024), but defines a particular correction in AdamW-preconditioned coordinates. The stabilized projection is only approximately orthogonal under conflict. Its geometry and budget precede multiplication by group learning rates and exclude decoupled weight decay. They therefore do not guarantee task-loss descent or preservation.

B SPARSITY ACCOUNTING

B.1 EXACT COUNTERS

Activation sparsity at site s is $\mathcal{Z}_s = \sum_{\ell,e,i} \mathbf{1}\{A_{s,i}^{(\ell,e)} = 0\} / \sum_{\ell,e} |A_s^{(\ell,e)}|$, where ℓ indexes layers, e evaluation batches, i tensor entries, and $|A|$ the number of elements in tensor A . Near-zero mass replaces equality by $|A_{s,i}| \leq \varepsilon$ for a stated tolerance; it does not contribute to exact-zero counts unless the value is zero. All aggregate fractions pool integer numerators and denominators across batches and layers before division.

The atomic unit of model-wide accounting is one scalar multiplication in a matrix product. If $X \in \mathbb{R}^{n \times p}$ feeds a linear projection with q outputs, the count is $N = npq$, with $N_0 = q \sum_{i,j} \mathbf{1}\{X_{ij} = 0\}$. Zero weights receive no credit. For the multihead attention graph specified in Appendix C.1, both operands are activations, so a product is credited once if either is zero. Within one block and evaluation batch, for query position t , key position $u \leq t$, and within-head coordinate j ,

$$N_{0,QK} = \sum_{b,h,t} \sum_{u \leq t} \sum_j \mathbf{1}\{Q_{bhtj} = 0 \vee K_{bhuj} = 0\}, \quad (2)$$

$$N_{0,PV} = \sum_{b,h,t} \sum_{u \leq t} \sum_j \mathbf{1}\{P_{bhtu} = 0 \vee V_{bhuj} = 0\}. \quad (3)$$

Here P is the causal softmax probability tensor; the model numerator sums these counts over blocks and evaluation batches. Future masked positions are excluded from both counts, while zeros on valid positions, including floating-point underflow, are included. Counts use the operands actually consumed by each matrix product; upstream masks do not determine those operands' zeros after intervening transformations. Earlier value positions participate in more causal pairs, so local value sparsity alone does not determine PV sparsity.

B.2 SPARSITY CEILING AND NORMALIZATION

For a specified graph and workload, let N_o count the valid scalar multiplications in block operation family o , pooled over layers and examples. Then $N_{\text{model}} = \sum_o N_o + N_{\text{head}}$, while N_0 sums the corresponding zero-operand counts only in block operations. The block-only view is $\mathcal{S}_{\text{block}} = N_0 / \sum_o N_o$. Both sparsities are fractions; percentages multiply them by 100. Embedding lookup, normalization, nonlinearities, softmax, RoPE, bias and residual additions, and data movement are outside this accounting.

To construct the reach numerator, count products guaranteed to have a zero activation operand as a structural consequence of 100% activation sparsity at the selected sites, independently of checkpoint values or natural zeros elsewhere. Include exact zero-preserving propagation under the declared graph rules. Products reached through more than one site receive one credit. The resulting count is $N_{\text{reach}}(\mathcal{N})$. Division by the same N_{model} gives $\mathcal{S}_{\text{model}}^{\text{max}}$. Natural zeros outside this selected-site reach can contribute to observed $\mathcal{S}_{\text{model}}$, so the ceiling is not an upper bound on every observation. Its gap to an observation does not measure sparsity available without quality loss. Appendix C.2 specializes the denominator and reach rules to the Pythia graph used in the experiments.

C EXPERIMENTAL SETUP DETAILS

C.1 PYTHIA SITES AND NONLINEARITIES

Pythia uses parallel attention and FFN branches with separate LayerNorms. For batch size B , sequence length T , residual width d , FFN width d_f , and H heads, let $d_h = d/H$. Table 3 identifies the actual operands recorded after the selected nonlinearity. The same site mapping is applied in every layer, with one common threshold per condition. Figure 2 shows the shared block; Table 1 gives its thresholding and pressure assignments.

At h , the selected nonlinearity replaces GELU. Other sites retain their original activations unless a thresholding nonlinearity is applied. At zero threshold, A7's additional symmetric query/key/value nonlinearities are exact identities in both values and gradients. They therefore do not change A4's

Table 3: Intervention sites and the matrix products that consume them directly.

Site	Shape	Consuming operation	Placement
a	$B \times T \times d$	QKV projection	After attention LayerNorm
m	$B \times T \times d$	FFN up projection	After FFN LayerNorm
h	$B \times T \times d_f$	FFN down projection	Nonlinearity output
q, k	$B \times H \times T \times d_h$	Attention scores	After partial RoPE
v	$B \times H \times T \times d_h$	Probability-value	After the QKV split
z	$B \times T \times d$	Attention output	Concatenated context, before output projection

mathematical forward map in the absence of pressure. A7-OL1(all) still differs from A4-OL1(all) in its pressure targets and their equal-tensor normalization.

Query and key counts use actual post-RoPE operands: rotation can repopulate an individual zero coordinate. Likewise, the output projection uses actual post-nonlinearity context z , rather than inferring its zeros from the value mask.

C.2 ARCHITECTURE-SPECIFIC COUNTS AND SPARSITY CEILINGS

The operation inventory comprises QKV projection, attention scores (QK), probability-value products (PV), attention-output projection, and the two FFN projections. For one uncached, unpadded sequence, each block contributes $3Td^2$ products from QKV projection, Td^2 from the attention-output projection, Tdd_f from each FFN projection, and $dT(T+1)/2$ from each of QK and PV . Thus, for L blocks and vocabulary size V_{vocab} ,

$$\begin{aligned} C_{\text{block}} &= T(4d^2 + 2dd_f) + dT(T+1), \\ C_{\text{model}} &= LC_{\text{block}} + TdV_{\text{vocab}}. \end{aligned} \quad (4)$$

Over B sequences, $N_{\text{model}} = BC_{\text{model}}$; the dense head contributes only to the denominator, as defined in Section 3.2.

Let $\mathcal{O}(\mathcal{N})$ be the union of operations reachable when the selected sites have 100% activation sparsity: a reaches QKV; m reaches FFN up; h reaches FFN down; either q or k reaches QK ; v reaches PV and attention output; and z reaches attention output. Each operation is counted once. The v rule includes the exact closure $V = 0 \Rightarrow PV = 0$. No further propagation is assumed: QKV biases can survive $a = 0$, and zero scores do not imply zero softmax probabilities. For the same site set in every layer, $N_{\text{reach}} = BL \sum_{o \in \mathcal{O}(\mathcal{N})} C_o$, where C_o is that operation’s per-sequence count. Division by N_{model} gives $\mathcal{S}_{\text{model}}^{\text{max}}$.

With Pythia’s $d_f = 4d$, the per-block reachable counts for A0, A1-H, A4, and A7 are 0, $4Td^2$, $12Td^2$, and C_{block} , respectively. The ladder evaluates these formulas at $T = 2,048$, $V_{\text{vocab}} = 50,304$, and the pinned (L, d) configurations (6, 128), (6, 512), and (24, 1,024).¹ No checkpoint enters this calculation. For example, A0 has zero selected-site reach but may have positive observed sparsity from natural zeros.

C.3 EVALUATION SCOPE

The original 54 endpoints pair quality and logical counts in the same eager-attention evaluation. For the ten 14M h-only endpoints, we retain the author-approved ordinary final-validation losses and separately evaluated logical counts from the same checkpoint. Both passes cover the complete validation set; across the 30 multisite 14M endpoints, their largest absolute loss difference is 0.000106 nats. Both loss versions and uniform-pass contrasts are retained in the supplementary pressure-placement data. Figure 1 uses ordinary reloaded final-checkpoint loss uniformly for all 44 trained checkpoints, including the ten added 70M h-only conditions, with separately evaluated canonical FP16 logical counts. Its post-hoc paths preserve the historical FP16 quality/count measurements. BF16 timing and qualification do not replace those validation losses. Figures 3 and 14 extend the

¹Architecture configurations: EleutherAI’s Pythia-14M-deduped, Pythia-70M-deduped, and Pythia-410M-deduped. Links identify immutable configuration revisions; released weights are not used.

same convention to all 74 trained models, with 410M shown in the appendix; small differences from the historical endpoint tables reflect the stated loss pass. The complete MiniPile validation split contains 500 documents. Concatenation and packing produce 338 complete blocks of 2,048 tokens (692,224 input tokens); the final 1,444-token tail is excluded. Each block contributes 2,047 next-token targets to cross-entropy, while product accounting includes all 2,048 input positions. Loss is averaged over the equally sized blocks, and sparsity is count-pooled. This workload uses no key-value cache. Condition-specific training budgets, optimizer settings, seeds, and checkpoint selection accompany their results; they are not implied by the ladder’s list of candidate conditions.

C.4 REALIZED TRAINING PROTOCOL

We use MiniPile and the Pythia-14M-deduped tokenizer for every size. Each document is encoded without added special tokens, followed by one end-of-sequence token. Concatenated tokens are packed into nonoverlapping 2,048-token blocks. Training uses a seeded permutation of the block indices; validation retains source order. The training split contains 1,000,000 documents and 1,491,711,416 tokens, yielding 728,374 complete blocks and an excluded 1,464-token tail. The supplementary `protocol.json` records immutable dataset and tokenizer revisions, cache hashes and implementation identities.

All conditions start from random weights drawn from the pinned Pythia architecture configurations; released pretrained weights are not used. Model initialization and data-order seeds are both 1234, with one training seed per condition. Parameter sweeps measure intervention variability rather than variation over independent training seeds. All quality comparisons reuse the same validation split, without a separate confirmation set. Within a size, the conditions share the initial parameter hash and training schedule. The 40-condition 14M cohort contains A0, A1-H, four weights each for A1-H-L1 and A1-H-OL1, and five thresholds for each of T_4/P_0 , T_4/P_h , T_4/P_4 , T_7/P_0 , T_7/P_h , and T_7/P_7 . Each larger cohort contains A0, A1-H, and the five-threshold T_4/P_4 and T_7/P_7 sweeps. The 70M extension adds five thresholds each for T_4/P_h and T_7/P_h , giving 22 conditions at 70M and 12 at 410M. Local pressure weights are $\{0.05, 0.1, 0.5, 1\}$; thresholds are $\{0, 0.01, 0.05, 0.1, 0.5\}$. Multisite OL1 uses $\lambda = 1$, with pressure at h only or at all thresholded sites. Every OL1 condition uses trust budget 1 and projection stabilizer $\epsilon = 10^{-12}$. The historical four-site h-only conditions are identified by their audited executed pressure capture, not their original all-site declaration. They are valid T_4/P_h conditions; the ten 14M h-only endpoints match the initialization, order and final-checkpoint protocol of the original cohort. The ten 70M h-only runs likewise match the original 70M initialization, data schedule, optimizer and token budget.

Training uses AdamW with $\beta = (0.9, 0.95)$, $\epsilon = 10^{-8}$, weight decay 0.1 and global task-gradient clipping at norm 1. Naive L1 clips the combined objective gradient instead. The learning rate warms up linearly for eight updates and follows cosine decay to one tenth of its peak. Parameters and optimizer state are FP32; forward computation uses FP16 autocast with dynamic loss scaling. Attention and hidden dropout are zero. Table 4 gives size-dependent settings. The global batch is 1,024 sequences of 2,048 tokens, for 712 optimizer updates and 1,493,172,224 input tokens. This wraps 714 blocks beyond the 728,374 complete blocks in one packed training pass. We use the final checkpoint without validation-based checkpoint selection. The lower 410M learning rate is part of the realized protocol, so size alone is not the only cross-scale change.

Table 4: Size-dependent training settings. Tokens per parameter use the common realized input-token budget; they are not held fixed across sizes.

Setting	14M	70M	410M
Parameters	14,067,712	70,426,624	405,334,016
Peak learning rate	10^{-3}	10^{-3}	3×10^{-4}
Minimum learning rate	10^{-4}	10^{-4}	3×10^{-5}
Microbatch (sequences)	32	4	4
Gradient accumulation	32	256	256
Tokens per parameter	106.14	21.20	3.68

Figure 7 shows the A0 optimization trajectories under the common token budget. Training loss trends downward across the runs, but the three sizes finish in different optimization regimes: 70M

reaches the lowest training loss under the realized schedule, while 410M finishes above it and exhibits a larger pre-clipping task-gradient norm late in training. These observations are **consistent with different degrees of optimization under the fixed-token protocol**, but they do not establish whether any run has or has not converged. The trajectories therefore provide context for the non-monotonic A0 validation losses in Figures 3 and 14 rather than a basis for extrapolating performance with additional training.

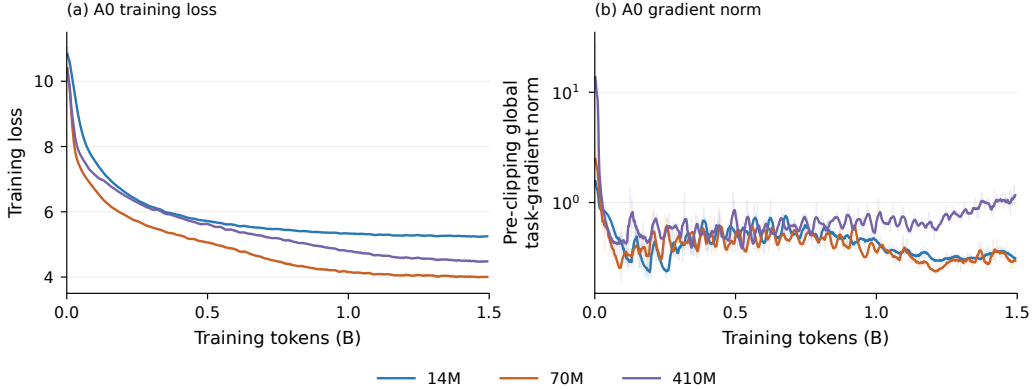


Figure 7: **A0 optimization trajectories under the common training-token budget.** (a) Training loss as a function of consumed input tokens for the 14M, 70M, and 410M A0 runs. (b) Pre-clipping global task-gradient norm over the same token budget; the logarithmic y-axis shows the large initial gradient transient and subsequent training regime. Faint traces show all 712 logged optimizer steps per size; opaque curves show a centered nine-step arithmetic moving mean, with partial windows at the edges. Norms are measured after loss unscaling and gradient accumulation, before clipping; smoothing is applied in the original units. All three sizes receive approximately 1.493B input tokens, but this corresponds to progressively fewer tokens per parameter as model size increases; the 410M run also uses a lower peak learning rate. Global norms are not normalized by parameter count. These trajectories characterize the realized optimization regimes and are not used as a convergence criterion.

D COMPLETE RESULTS AND SUPPORTING DIAGNOSTICS

The following tables report the original 64 trained endpoints and the ten additional 70M h-only endpoints, the 29 original paired 14M contrasts, 25 additional pressure-scope contrasts, and all 15 cross-scale broad-pressure recipe comparisons. They retain dominated settings as well as favorable ones. Each entry summarizes one trained model on complete validation. Sparsity percentages are formed from pooled integer product counts. The endpoint table uses the common seven-site sparsity-ceiling denominator from Table 1, rather than a different denominator for each recipe. Here $U_{\text{arch}} = S_{\text{model}}/S_{\text{model}}^{\text{max}}$ (A7) denotes this common-reference ratio, retained for the complete endpoint table.

Table 5: The original 64 trained endpoints; Table 6 adds the ten 70M h-only conditions. T_i/P_j specifies threshold and pressure scope; P_0 means no pressure. U_{arch} uses the common seven-site ceiling within each size. H-only losses use ordinary final validation; other losses use the logical diagnostic pass (Appendix C.3). Rounded zero sparsity need not be exactly zero.

Size	Recipe	Parameter	Loss	S_{model} (%)	U_{arch} (%)
14M	A0	—	5.2086	0.000	0.000
14M	A1-H	—	5.2696	2.714	9.061
14M	A1-H-L1	$\lambda = 0.05$	5.2061	3.142	10.489
14M	A1-H-L1	$\lambda = 0.1$	5.1655	3.336	11.139
14M	A1-H-L1	$\lambda = 0.5$	5.1127	3.788	12.645
14M	A1-H-L1	$\lambda = 1$	5.1023	3.949	13.185
14M	A1-H-OL1	$\lambda = 0.05$	5.1981	3.145	10.501

Table 5 (continued)

Size	Recipe	Parameter	Loss	$\mathcal{S}_{\text{model}}$ (%)	U_{arch} (%)
14M	A1-H-OL1	$\lambda = 0.1$	5.1594	3.339	11.146
14M	A1-H-OL1	$\lambda = 0.5$	5.1102	3.768	12.580
14M	A1-H-OL1	$\lambda = 1$	5.1212	3.938	13.149
14M	T_4/P_0	$\kappa = 0$	5.4705	7.212	24.078
14M	T_4/P_0	$\kappa = 0.01$	5.4665	7.414	24.752
14M	T_4/P_0	$\kappa = 0.05$	5.4341	8.206	27.396
14M	T_4/P_0	$\kappa = 0.1$	5.4197	8.953	29.891
14M	T_4/P_0	$\kappa = 0.5$	5.6597	10.216	34.106
14M	T_4/P_h	$\kappa = 0$	5.2157	8.409	28.076
14M	T_4/P_h	$\kappa = 0.01$	5.2045	8.586	28.665
14M	T_4/P_h	$\kappa = 0.05$	5.1956	9.036	30.167
14M	T_4/P_h	$\kappa = 0.1$	5.2287	9.343	31.194
14M	T_4/P_h	$\kappa = 0.5$	5.7227	10.227	34.146
14M	T_4/P_4	$\kappa = 0$	5.4583	7.810	26.075
14M	T_4/P_4	$\kappa = 0.01$	5.4583	8.587	28.668
14M	T_4/P_4	$\kappa = 0.05$	5.4898	10.456	34.910
14M	T_4/P_4	$\kappa = 0.1$	5.5483	11.394	38.041
14M	T_4/P_4	$\kappa = 0.5$	6.0380	12.713	42.446
14M	T_7/P_0	$\kappa = 0$	5.4684	7.218	24.097
14M	T_7/P_0	$\kappa = 0.01$	5.4588	7.618	25.433
14M	T_7/P_0	$\kappa = 0.05$	5.4379	9.127	30.471
14M	T_7/P_0	$\kappa = 0.1$	5.4287	10.425	34.805
14M	T_7/P_0	$\kappa = 0.5$	5.7029	15.387	51.371
14M	T_7/P_h	$\kappa = 0$	5.2198	8.399	28.042
14M	T_7/P_h	$\kappa = 0.01$	5.1981	8.856	29.567
14M	T_7/P_h	$\kappa = 0.05$	5.1950	10.126	33.807
14M	T_7/P_h	$\kappa = 0.1$	5.2374	10.904	36.403
14M	T_7/P_h	$\kappa = 0.5$	5.7320	16.664	55.634
14M	T_7/P_7	$\kappa = 0$	5.4802	7.054	23.551
14M	T_7/P_7	$\kappa = 0.01$	5.4758	7.723	25.786
14M	T_7/P_7	$\kappa = 0.05$	5.4628	9.863	32.929
14M	T_7/P_7	$\kappa = 0.1$	5.4295	11.797	39.385
14M	T_7/P_7	$\kappa = 0.5$	5.8294	27.483	91.755
70M	A0	—	4.0998	0.000	0.001
70M	A1-H	—	4.2227	10.066	20.366
70M	T_4/P_4	$\kappa = 0$	4.8054	25.673	51.944
70M	T_4/P_4	$\kappa = 0.01$	4.8229	26.224	53.060
70M	T_4/P_4	$\kappa = 0.05$	4.8742	28.490	57.644
70M	T_4/P_4	$\kappa = 0.1$	4.9439	30.574	61.861
70M	T_4/P_4	$\kappa = 0.5$	5.3895	35.596	72.022
70M	T_7/P_7	$\kappa = 0$	4.9412	23.562	47.674
70M	T_7/P_7	$\kappa = 0.01$	4.9364	24.226	49.017
70M	T_7/P_7	$\kappa = 0.05$	4.9637	26.524	53.666
70M	T_7/P_7	$\kappa = 0.1$	4.9501	28.694	58.056
70M	T_7/P_7	$\kappa = 0.5$	5.2159	40.602	82.150
410M	A0	—	4.5475	0.011	0.013
410M	A1-H	—	4.6513	21.050	24.127
410M	T_4/P_4	$\kappa = 0$	5.6921	38.339	43.944
410M	T_4/P_4	$\kappa = 0.01$	5.6914	39.149	44.873
410M	T_4/P_4	$\kappa = 0.05$	5.6789	42.125	48.283
410M	T_4/P_4	$\kappa = 0.1$	5.6789	45.688	52.367
410M	T_4/P_4	$\kappa = 0.5$	5.1910	71.591	82.058
410M	T_7/P_7	$\kappa = 0$	5.4263	41.121	47.133
410M	T_7/P_7	$\kappa = 0.01$	5.4331	41.478	47.542
410M	T_7/P_7	$\kappa = 0.05$	5.4679	44.053	50.494
410M	T_7/P_7	$\kappa = 0.1$	5.4907	48.998	56.161
410M	T_7/P_7	$\kappa = 0.5$	5.1207	80.616	92.401

D.1 ACTIVATION SPARSITY AND SIGNED-DENSITY COVERAGE

The seven frozen-checkpoint evaluations in Figure 10 record signed values after the trained nonlinearities, with Q/K measured after RoPE. Each checkpoint contributes 2,658,140,160 FFN elements

Table 6: Additional 70M h-only endpoints in Figure 1. Loss uses ordinary reloaded final validation; sparsity uses canonical FP16 logical counts. U_{arch} uses the common seven-site ceiling, as in Table 5. Together the two tables cover all 74 trained conditions.

Recipe	κ	Loss	$\mathcal{S}_{\text{model}}$ (%)	U_{arch} (%)
T_4/P_h	0	4.6065	25.792	52.185
T_4/P_h	0.01	4.6185	25.971	52.547
T_4/P_h	0.05	4.5964	26.566	53.752
T_4/P_h	0.1	4.7020	27.218	55.071
T_4/P_h	0.5	5.3057	29.236	59.153
T_7/P_h	0	4.6508	25.786	52.173
T_7/P_h	0.01	4.6122	26.009	52.624
T_7/P_h	0.05	4.6670	26.895	54.416
T_7/P_h	0.1	4.7189	27.779	56.206
T_7/P_h	0.5	5.3372	32.233	65.216

Table 7: All 29 Pythia-14M contrasts, treatment minus reference. When both recipes have a swept parameter, its value is matched; A0 and A1-H are fixed controls.

Treatment – reference	Parameter	Δ loss	$\Delta\mathcal{S}_{\text{model}}$ (pp)
A1-H – A0	—	+0.0611	+2.714
A1-H-L1 – A1-H	$\lambda = 0.05$	-0.0635	+0.427
A1-H-L1 – A1-H	$\lambda = 0.1$	-0.1042	+0.622
A1-H-L1 – A1-H	$\lambda = 0.5$	-0.1569	+1.073
A1-H-L1 – A1-H	$\lambda = 1$	-0.1674	+1.235
A1-H-OL1 – A1-H-L1	$\lambda = 0.05$	-0.0081	+0.004
A1-H-OL1 – A1-H-L1	$\lambda = 0.1$	-0.0061	+0.002
A1-H-OL1 – A1-H-L1	$\lambda = 0.5$	-0.0025	-0.020
A1-H-OL1 – A1-H-L1	$\lambda = 1$	+0.0189	-0.011
A4 – A1-H	$\kappa = 0$	+0.2009	+4.498
A4 – A1-H	$\kappa = 0.01$	+0.1969	+4.700
A4 – A1-H	$\kappa = 0.05$	+0.1645	+5.492
A4 – A1-H	$\kappa = 0.1$	+0.1500	+6.239
A4 – A1-H	$\kappa = 0.5$	+0.3900	+7.501
A4-OL1(all) – A4	$\kappa = 0$	-0.0122	+0.598
A4-OL1(all) – A4	$\kappa = 0.01$	-0.0082	+1.173
A4-OL1(all) – A4	$\kappa = 0.05$	+0.0556	+2.251
A4-OL1(all) – A4	$\kappa = 0.1$	+0.1286	+2.441
A4-OL1(all) – A4	$\kappa = 0.5$	+0.3783	+2.498
A7 – A4	$\kappa = 0$	-0.0021	+0.006
A7 – A4	$\kappa = 0.01$	-0.0077	+0.204
A7 – A4	$\kappa = 0.05$	+0.0038	+0.921
A7 – A4	$\kappa = 0.1$	+0.0090	+1.472
A7 – A4	$\kappa = 0.5$	+0.0432	+5.171
A7-OL1(all) – A7	$\kappa = 0$	+0.0118	-0.163
A7-OL1(all) – A7	$\kappa = 0.01$	+0.0170	+0.106
A7-OL1(all) – A7	$\kappa = 0.05$	+0.0249	+0.736
A7-OL1(all) – A7	$\kappa = 0.1$	+0.0008	+1.372
A7-OL1(all) – A7	$\kappa = 0.5$	+0.1265	+12.096

Table 8: Pressure-scope increments and threshold-placement contrasts at 14M. Each row subtracts the reference at the same κ . These are single-seed comparisons.

Treatment – reference	κ	ΔL	$\Delta \mathcal{S}_{\text{model}}$ (pp)
$T_4/P_h - T_4/P_0$	0	-0.2548	+1.197
$T_4/P_h - T_4/P_0$	0.01	-0.2620	+1.172
$T_4/P_h - T_4/P_0$	0.05	-0.2385	+0.830
$T_4/P_h - T_4/P_0$	0.1	-0.1910	+0.390
$T_4/P_h - T_4/P_0$	0.5	+0.0630	+0.012
$T_4/P_4 - T_4/P_h$	0	+0.2425	-0.599
$T_4/P_4 - T_4/P_h$	0.01	+0.2538	+0.001
$T_4/P_4 - T_4/P_h$	0.05	+0.2942	+1.421
$T_4/P_4 - T_4/P_h$	0.1	+0.3196	+2.051
$T_4/P_4 - T_4/P_h$	0.5	+0.3153	+2.486
$T_7/P_h - T_7/P_0$	0	-0.2486	+1.182
$T_7/P_h - T_7/P_0$	0.01	-0.2607	+1.238
$T_7/P_h - T_7/P_0$	0.05	-0.2429	+0.999
$T_7/P_h - T_7/P_0$	0.1	-0.1913	+0.479
$T_7/P_h - T_7/P_0$	0.5	+0.0292	+1.277
$T_7/P_7 - T_7/P_h$	0	+0.2604	-1.345
$T_7/P_7 - T_7/P_h$	0.01	+0.2777	-1.133
$T_7/P_7 - T_7/P_h$	0.05	+0.2678	-0.263
$T_7/P_7 - T_7/P_h$	0.1	+0.1921	+0.893
$T_7/P_7 - T_7/P_h$	0.5	+0.0974	+10.819
$T_7/P_h - T_4/P_h$	0	+0.0041	-0.010
$T_7/P_h - T_4/P_h$	0.01	-0.0064	+0.270
$T_7/P_h - T_4/P_h$	0.05	-0.0006	+1.090
$T_7/P_h - T_4/P_h$	0.1	+0.0087	+1.560
$T_7/P_h - T_4/P_h$	0.5	+0.0094	+6.436

Table 9: All matched A7-OL1(all) minus A4-OL1(all) comparisons. Negative loss changes and positive sparsity changes favor A7-OL1(all).

Size	κ	Δloss	$\Delta \mathcal{S}_{\text{model}}$ (pp)
14M	0	+0.0219	-0.756
14M	0.01	+0.0175	-0.863
14M	0.05	-0.0270	-0.593
14M	0.1	-0.1188	+0.402
14M	0.5	-0.2086	+14.769
70M	0	+0.1358	-2.110
70M	0.01	+0.1135	-1.998
70M	0.05	+0.0895	-1.966
70M	0.1	+0.0062	-1.880
70M	0.5	-0.1736	+5.006
410M	0	-0.2658	+2.783
410M	0.01	-0.2582	+2.329
410M	0.05	-0.2111	+1.929
410M	0.1	-0.1881	+3.310
410M	0.5	-0.0703	+9.024

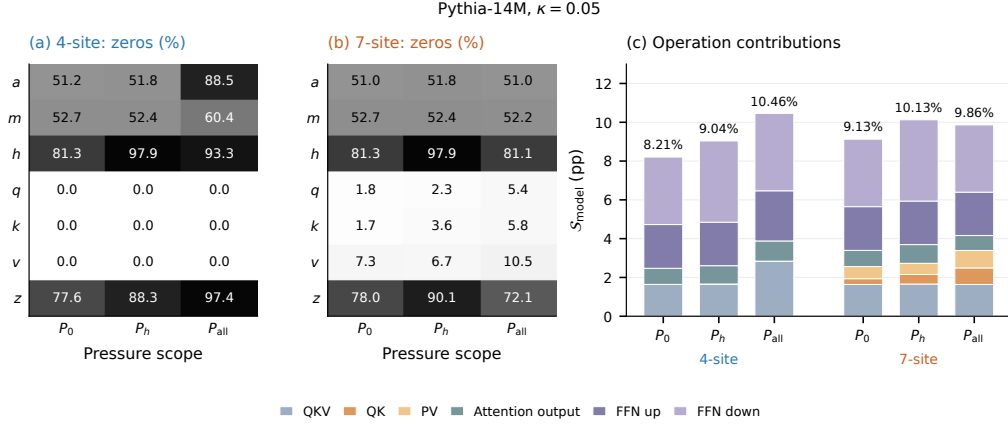


Figure 8: **Pressure-scope effects at the moderate threshold $\kappa = 0.05$.** The six 14M recipes use pooled counts and the full-model operation denominator defined in Appendix B. H-only pressure concentrates zeros at h while preserving a different pattern at m and Q/K/V from broad pressure. Rounded percentages do not establish all-zero or fully dense tensors.

and 1,594,884,096 attention elements across the complete validation set. Summing integer counts gives the 80:20 weighting of $h : m$ and equal weighting of $q : k : v$. These pooled distributions are element-weighted mixtures, not distributions of layer means.

Exact zeros are stored separately from the nonzero histogram. The native grid spans $[-8, 8]$ with 16,000 bins of nominal width 0.001; the figure combines ten bins at a time and divides by the actual floating-point bin width and the count of all captured elements. Consequently, a curve’s full integral is the recorded nonzero mass inside that grid, not one. Table 10 reports the zero mass and the fractions outside the displayed and native ranges. Underflow and overflow counts are retained rather than folded into edge bins. The displayed limits are $[-2.5, 3]$ for FFN and $[-4, 4]$ for attention in Figure 10. The largest omitted display tail is 6.6719%; the native-grid tail is at most 0.7607%. These historical densities cover no h-only multisite checkpoints. Figures 8 and 9 use retained final-validation site counters for all six threshold/pressure combinations, separating m and h . These counts were measured independently of the signed histogram reruns; small pass-to-pass rounding differences remain visible. The retained logical-product counters separately decompose model-wide sparsity into operation contributions at $\kappa = 0.05$ and 0.5.

The diagnostic reruns use FP32 parameters with FP16 autocast and no additional thresholding. Their maximum absolute validation-loss difference from the retained reference is 6.89×10^{-5} , below the specified 5×10^{-4} tolerance; all 108 threshold-consistency checks pass. Small rounding differences in exact-zero counts between evaluation passes do not change the displayed conclusions. Complete per-site and pooled histograms, counts and checkpoint hashes accompany the data release.

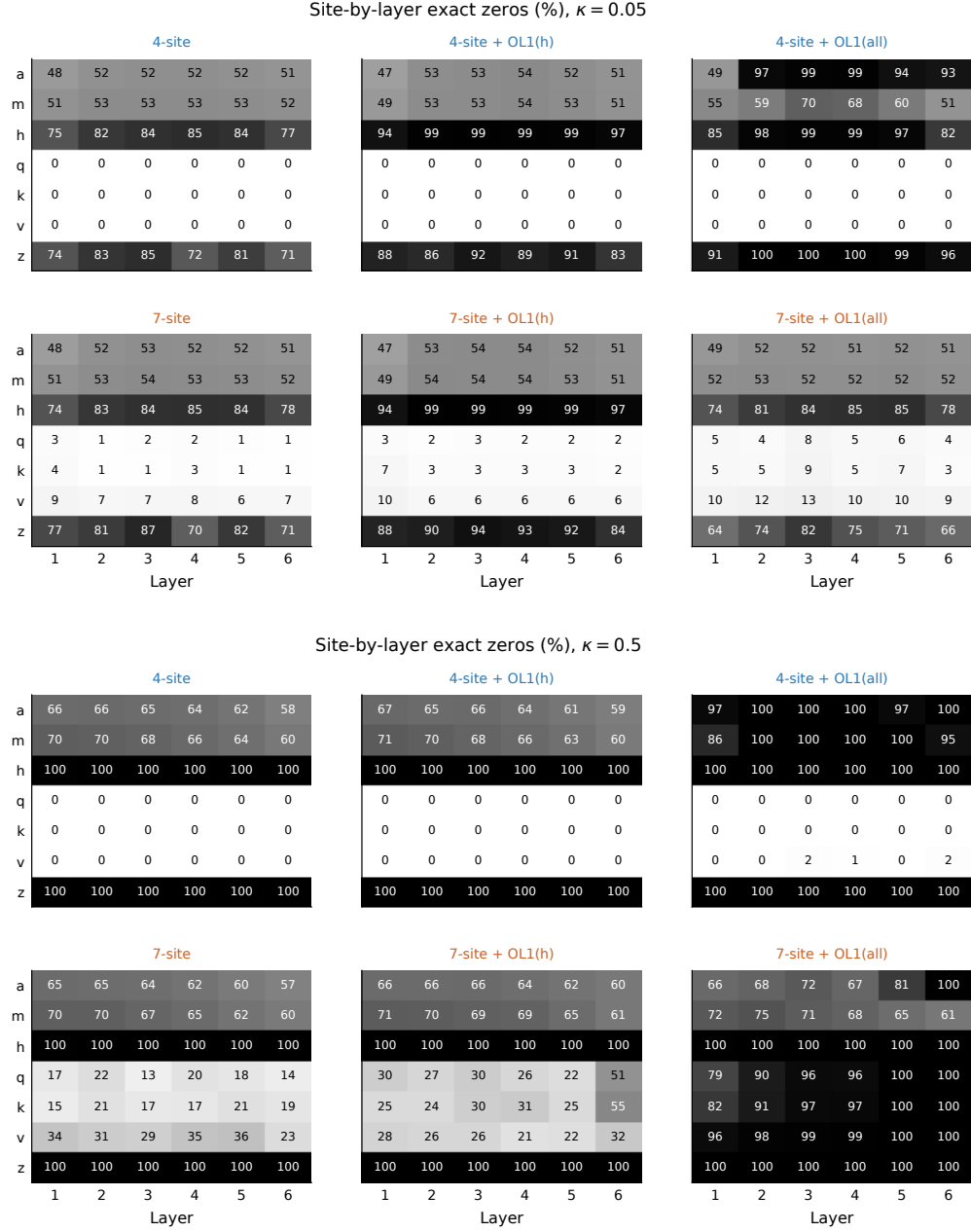


Figure 9: **Site-by-layer zero fractions under matched pressure scopes.** The six recipes at $\kappa = 0.05$ (top) and 0.5 (bottom). Each cell pools integer counts over all 338 validation blocks at one site and layer. Q/K are post-RoPE; values are rounded to whole percentages, so a displayed 100 does not imply an all-zero tensor. The complete five-threshold counters and source identities accompany the supplementary data.

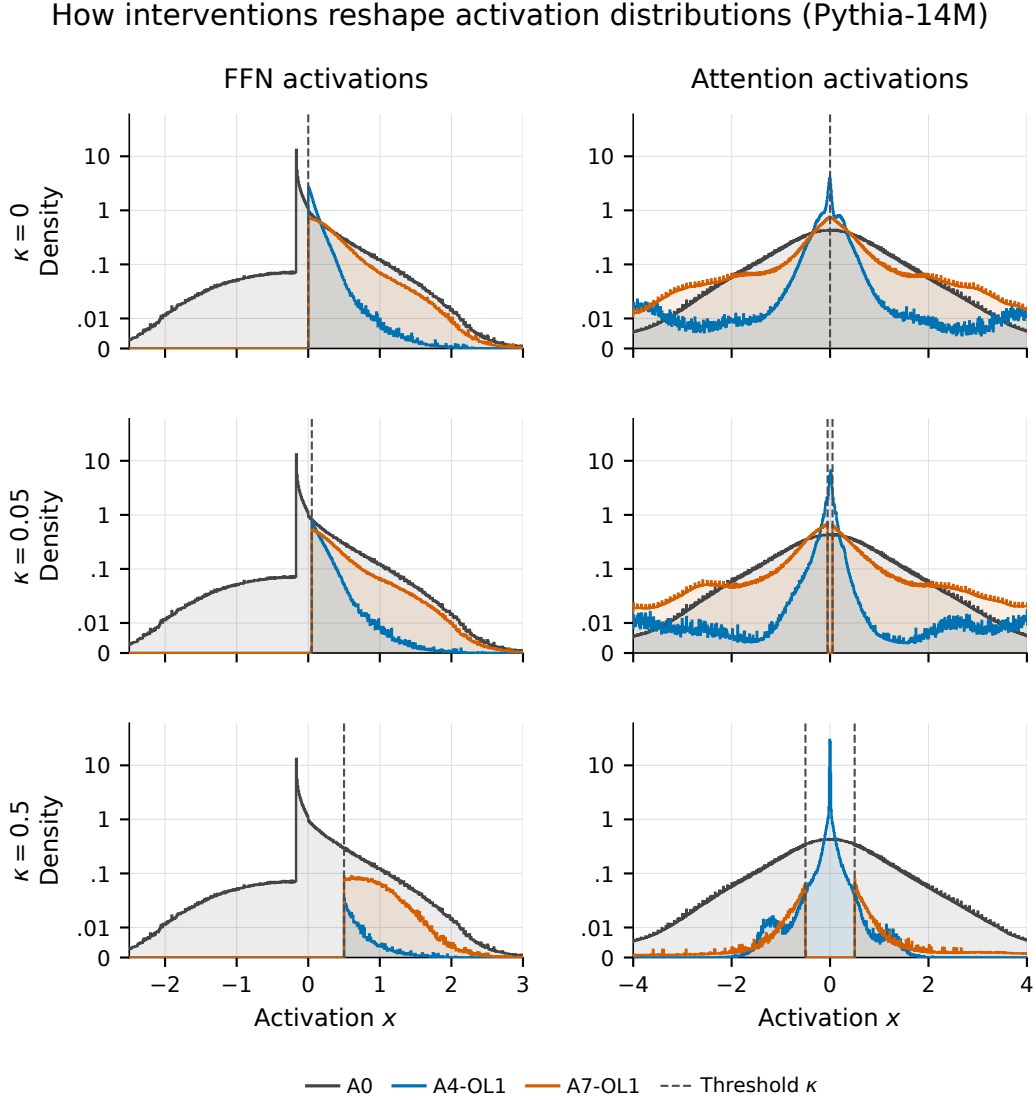


Figure 10: **Interventions reshape FFN and attention distributions differently. Exact zeros are omitted from the curves and reported in Table 10.** Columns pool FFN (h, m) and attention (q, k, v) activations; rows use trained thresholds $\kappa = 0, 0.05, 0.5$. A0 is repeated across rows. Counts are pooled across all layers and validation blocks, weighting $h : m$ by 80:20 and $q : k : v$ equally. Density divides nonzero bin counts by all captured elements and bin width, without renormalizing the visible range. The vertical scale is symlog, linear below 0.01; filled areas on this scale do not represent probability mass. Dashed lines mark $+\kappa$ for A4/A7 FFN thresholds and $\pm\kappa$ for A7 attention thresholds. Q/K are measured after RoPE. A4 leaves q, k, v without direct thresholding or pressure; A7 targets them directly. The seven checkpoints use no additional post-hoc thresholding.

Table 10: Activation sparsity (the probability mass at zero) omitted from the density curves, and mass outside their displayed range (view tail) and the native $[-8, 8]$ histogram (grid tail). Grid tails are a subset of view tails, not an additional mass to add. All percentages divide integer counts by all captured elements. A0 is listed once although it is repeated in each figure row. These 14 groups come from seven checkpoints.

Recipe	κ	Group	Sparsity (%)	View tail (%)	Grid tail (%)
A0	—	FFN	< 0.01	0.1113	0.0000
A0	—	Attention	< 0.01	2.3929	0.4632
A4-OL1(all)	0	FFN	67.22	0.0003	0.0000
A4-OL1(all)	0	Attention	< 0.01	5.1717	0.6162
A4-OL1(all)	0.05	FFN	86.71	0.0003	0.0000
A4-OL1(all)	0.05	Attention	< 0.01	5.6601	0.7607
A4-OL1(all)	0.5	FFN	99.31	0.0001	0.0000
A4-OL1(all)	0.5	Attention	0.22	0.0003	0.0000
A7-OL1(all)	0	FFN	64.52	0.0063	0.0000
A7-OL1(all)	0	Attention	< 0.01	5.4657	0.4648
A7-OL1(all)	0.05	FFN	75.29	0.0106	0.0000
A7-OL1(all)	0.05	Attention	7.26	6.6719	0.6932
A7-OL1(all)	0.5	FFN	93.64	0.0037	0.0000
A7-OL1(all)	0.5	Attention	95.60	0.2396	0.0009

D.2 OPERATION CONTRIBUTIONS TO MODEL-WIDE SPARSITY

An operation’s contribution to S_{model} is its zero-product count divided by the full model product count. Summing these contributions recovers the model-wide fraction; summing local activation percentages would not. Table 11 gives this decomposition at $\kappa = 0.5$. At 14M, extending from A4-OL1(all) to A7-OL1(all) adds attention opportunity while reducing the contribution from some projections. At larger sizes, projection work carries more weight in the workload, so an equal change in activation sparsity need not yield an equal change in model-wide sparsity. Figure 11 displays the operation-level view at the same threshold and all three sizes.

Table 11: Contributions to $100S_{\text{model}}$ in percentage points at $\kappa = 0.5$. All six operations share the same model denominator within each size. The dense LM head contributes to that denominator but receives no zero credit. Differences in rounded entries can affect the displayed sum.

Operation	14M		70M		410M	
	A4-OL1(all)	A7-OL1(all)	A4-OL1(all)	A7-OL1(all)	A4-OL1(all)	A7-OL1(all)
QKV	3.173	2.430	8.820	6.231	17.563	16.992
QK	0.000	8.324	0.000	5.052	0.000	5.206
PV	0.057	8.449	0.011	5.371	0.008	5.858
Attention output	1.069	1.069	3.087	3.084	6.186	6.187
FFN up	4.139	2.939	11.329	8.526	22.938	21.503
FFN down	4.275	4.272	12.350	12.338	24.895	24.869
Total	12.713	27.483	35.596	40.602	71.591	80.616

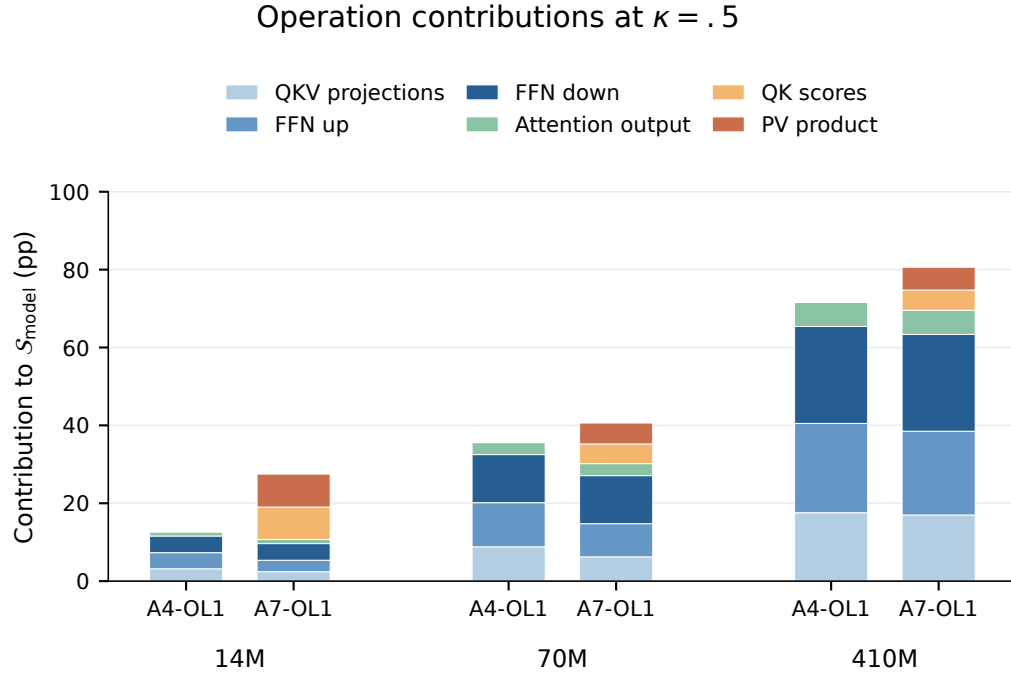


Figure 11: Operation accounting for A4-OL1(all) and A7-OL1(all) at $\kappa = 0.5$ across 14M, 70M and 410M. Zero-operand products are counted over causally valid positions and pooled across all 338 validation blocks. Each contribution uses the full model denominator, including the dense LM head without zero credit. At 14M, attention contributes more under A7-OL1(all), outweighing smaller contributions from several projections. The bars show logical multiplication opportunities, not runtime savings.

D.3 COMPLETE HISTORICAL POST-HOC THRESHOLDING TRAJECTORIES

Historical plot legends retain “clipping” for post-hoc thresholding and “A4-OL1/A7-OL1” for the all-site pressure recipes. They do not include the newly integrated h-only conditions. The historical post-hoc release contains 540 evaluations: ten thresholding targets for each of the original 54 trained checkpoints. The ten newly included h-only endpoints have no corresponding post-hoc sweep. The release joins 350 evaluations with 190 retained evaluations matched by checkpoint content hash. Calibration uses the first ten complete training blocks in source order, separately for each layer’s a, m, h, z activations. Evaluation zeros values satisfying $|x| \leq t$ after the trained nonlinearities, without changing weights or A7’s internal attention nonlinearities. The target p controls a calibration quantile; it is not the achieved model-wide sparsity. In particular, an existing zero mass can make several targets identical.

Figures 12–15 retain the full loss range and every target, including dominated points and ties. Each path begins at its actually measured $p = 0$ evaluation. Small numerical differences from the canonical training endpoint can change strict nondominance without implying a substantive improvement. Numerical frontier memberships are stored separately from the lines connecting evaluated settings. All evaluations use complete validation with FP32 parameters, FP16 autocast, eager attention, batch one and $T = 2,048$.

14M training and post-hoc clipping

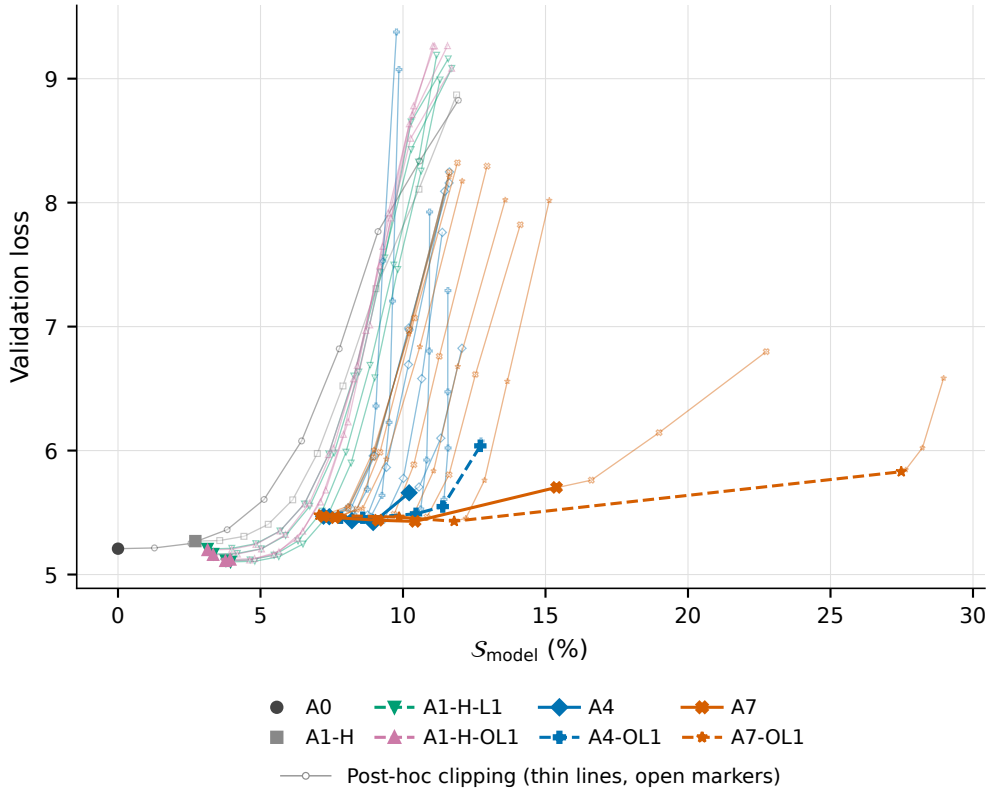


Figure 12: Complete 14M training and post-hoc thresholding results: 30 trained endpoints and 300 post-hoc thresholding evaluations. Large filled markers denote trained endpoints; bold lines connect their settings, solid for unpressured recipes and dashed for L1/OL1. Each faint open-marker path follows one fixed checkpoint through $p = 0, 0.1, \dots, 0.9$. Colors and symbols identify its source recipe. The full loss range is shown. Connections order evaluated settings and do not interpolate attainable models or fitted Pareto envelopes.

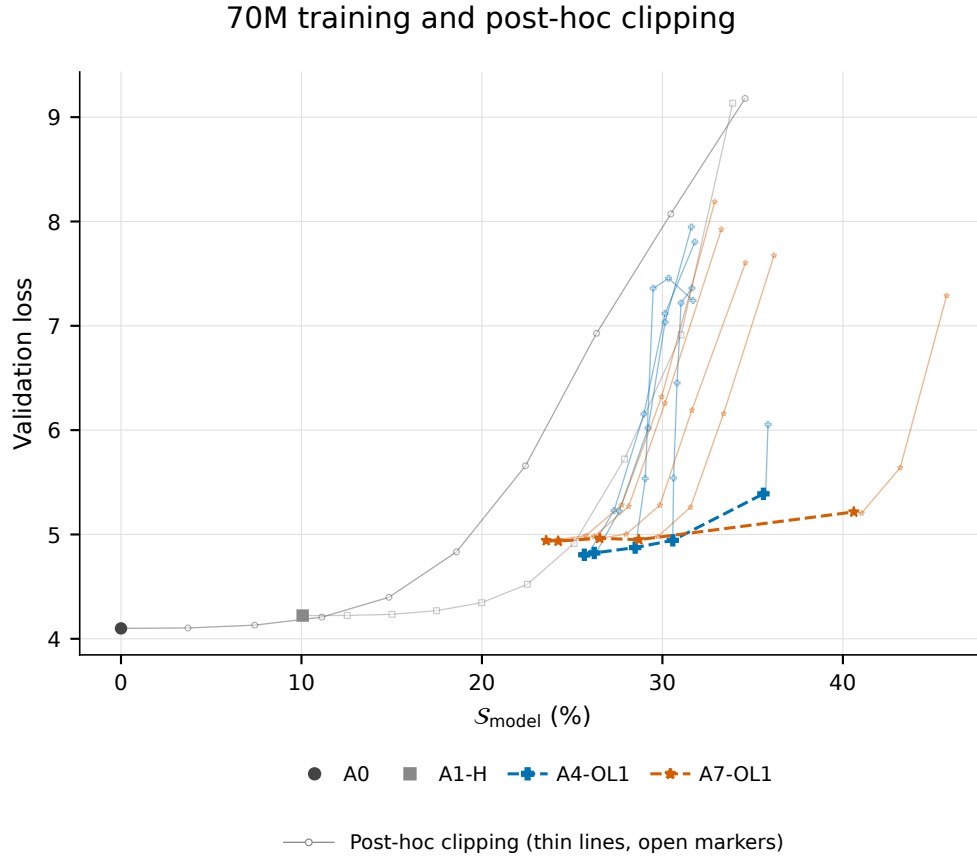


Figure 13: Complete 70M training and post-hoc thresholding results: 12 trained endpoints and 120 post-hoc thresholding evaluations. Large filled markers and bold lines identify the trained recipes; faint open-marker paths follow each checkpoint through ten increasing thresholding targets. All points, including overlaps and high-loss settings, are retained. The evaluation and line conventions match Figure 12.

410M as a fixed-token stress test. Figure 14 retains all twelve trained 410M conditions and both complete control-clipping paths. At $\kappa = 0.5$, T_7/P_{all} reaches 80.62% model-wide sparsity, or 92.4% of the seven-site ceiling, at loss 5.121 versus the base model’s 4.547. This tests attainable sparsity under a larger architecture and the same token budget; it is not a quality-scaling experiment. The 410M seven-site ceiling is 87.25%, compared with 49.42% at 70M, so greater architectural reach also contributes to the larger observed model-wide sparsity.

The 410M base-model loss exceeds 70M’s 4.100. All sizes receive 712 optimizer steps and 1.493B input tokens, corresponding to 106.14, 21.20 and 3.68 tokens per parameter at 14M, 70M and 410M. The 410M schedule also has a lower peak learning rate (3×10^{-4} rather than 10^{-3}). These differences may contribute to the non-monotonic quality ordering, but their effects are not isolated. The training trajectories are retained in Figure 7. We therefore treat 410M as a limited stress test and use 14M/70M for detailed intervention and runtime analysis.

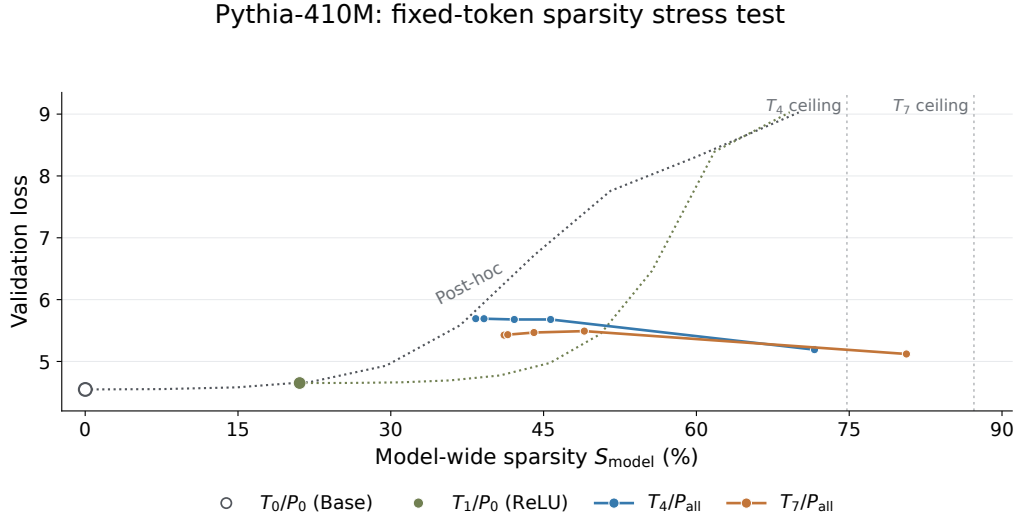


Figure 14: **Complete 410M quality–sparsity stress-test panel.** All twelve trained conditions: Base, ReLU, and T_4/P_{all} and T_7/P_{all} at $\kappa = 0, .01, .05, .1, .5$. Dotted paths retain all ten post-hoc targets for each fixed Base/ReLU control, including their full loss range. Vertical guides mark the four- and seven-site sparsity ceilings. Trained losses use ordinary final-checkpoint validation; clipping retains its original FP16 pass. Sparsity pools logical counts over all 338 complete validation blocks. Figure 15 additionally retains the historical clipping sweeps of every trained recipe.

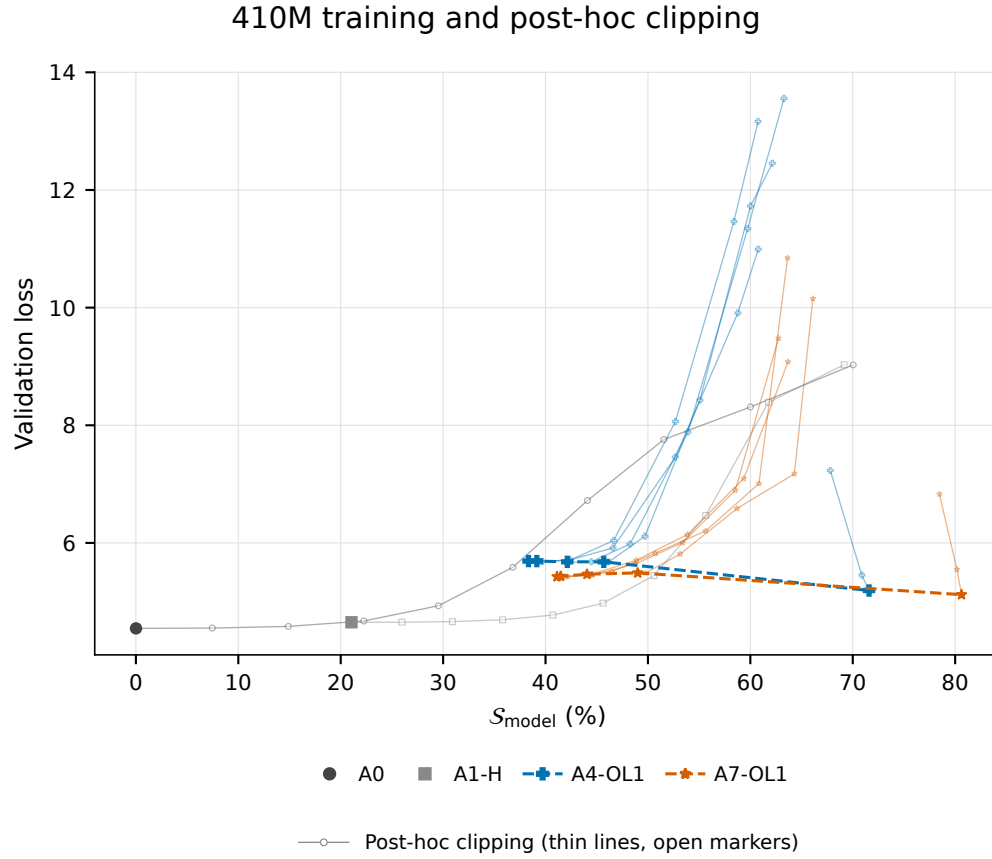


Figure 15: Complete 410M training and post-hoc thresholding results: 12 trained endpoints and 120 post-hoc thresholding evaluations. Large filled markers and bold lines identify the trained recipes; faint open-marker paths follow each checkpoint through ten increasing thresholding targets. All points, including overlaps and high-loss settings, are retained. The evaluation and line conventions match Figure 12.

D.4 KERNEL QUALIFICATION AND ABLATIONS

The unchanged Sakana/TwELL control achieves $1.3001\times$ on its supported SparseLM-0.5B/H100 workload. Its raw projection primitive at the Pythia-14M width $N = 128$ instead gives relative L2 error 0.69055, exceeding the 0.02 bound; that test stops before full-model Pythia timing. This is a compatibility test, not an unchanged upstream speedup comparison on our model. The later P0 adapter changes signed-value packing, shape tails and numerical handling for Pythia. It retains dense attention and uses native projection fallback on A0. Table 12 reports this adapted baseline.

Historical proposals are replayed on four representative checkpoints; the incumbent in Figure 16 uses fixed A7-OL1(all), $\kappa = 0.5$. It reaches $1.6024\times$ by iteration 11 and $1.7830\times$ at iteration 42. Final implementation and ablation summaries retain their original 30-checkpoint cohort. The main absolute-latency panel additionally restores the five qualified T_4/P_h checkpoints from the same historical 35-checkpoint sweep; they implement a valid, distinct pressure scope. These are existing measurements, reaggregated from raw timing pairs with the same geometric-mean rule. Figure 1 adds five measured T_7/P_h full-model timings at 14M, 22 trained models at 70M, and 40 control-clipping settings from separate RTX5090 sessions. They do not extend the historical skipping-ablation cohort. Search progress and the final sweep are separate measurements, explaining the small difference between their $1.7830\times$ and $1.7832\times$ maxima.

Sparse kernel auto-research for full-model acceleration

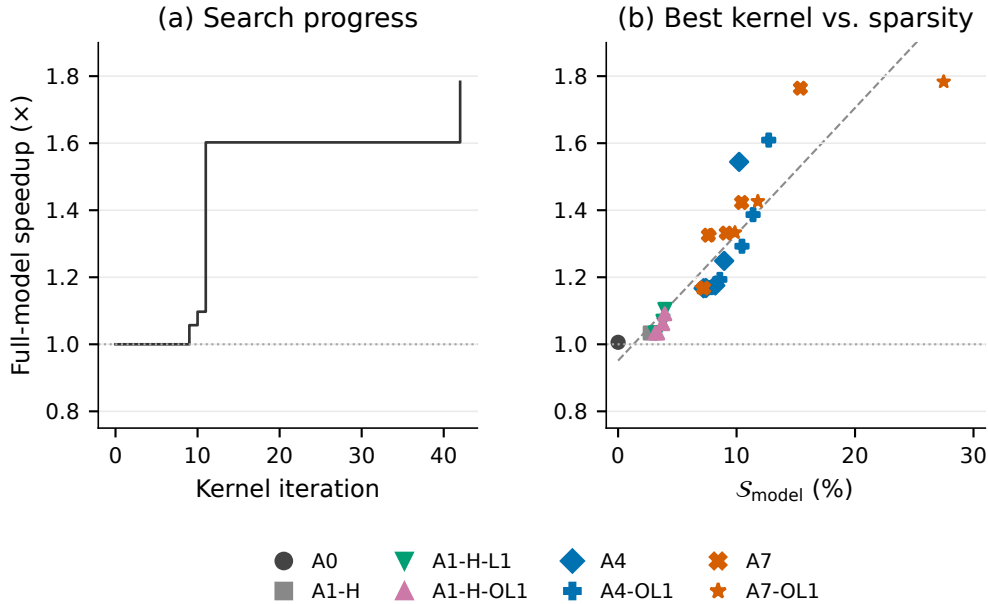


Figure 16: **Kernel-search history and the final checkpoint sweep.** (a) Best qualified incumbent across 42 eligible retrospective proposals on the fixed seven-site + OL1(all), $\kappa = 0.5$ checkpoint. (b) Final K050 speedup across the 30-checkpoint cohort; this historical view distinguishes individual recipes. Both use the same native PyTorch/SDPA CUDA-graph reference and BF16, batch-one, 2,048-token protocol on one RTX5090. Search replay and final evaluation are separate timing measurements. The history describes one human-guided development trajectory, not a comparison of coding agents.

The coding-agent configuration records *gpt-6-astra* as the default model, not the model served for every request. This is one human-guided development trajectory and does not compare agent-assisted with human-only kernel development. Kernel measurements use Python 3.12.3, PyTorch 2.11.0+cu128, Transformers 5.12.1, CUDA 12.8 and nvcc 12.8.93. The supplementary protocol records dependency and frozen-source hashes.

Every candidate must produce finite outputs, with each logit satisfying $|y - \hat{y}| \leq 0.25 + 0.02|\hat{y}|$, per-block relative output L2 error at most 0.02, and absolute pooled loss difference at most 0.001. Here $\hat{y}$ is the unmodified native eager reference. All three fresh processes must pass on all 338 validation blocks. Qualification is bounded numerical agreement on these inputs, not bitwise equivalence. Failed candidates are excluded from qualified speedup summaries. P0, an early projection adapter derived from the released kernels of Cetin et al. (2026), qualifies on only four checkpoints. Its average is not comparable to a 30-checkpoint mean or an unchanged upstream benchmark.

Timing uses the same checkpoint, inputs and CUDA-graph scaffold for native PyTorch/SDPA and each candidate. For each checkpoint, we take the geometric mean of 1,344 paired host-latency ratios (native divided by candidate): 64 inputs, seven randomized paired passes, and three fresh processes. Compilation, static weight preparation, graph capture and equal input staging are excluded. Preprocessing that depends on activations, embeddings, every block, final normalization and full vocabulary logits are included. There is no activation cache. Cohort summaries give each checkpoint equal weight.

K050 combines normalization and RoPE fusion, zero-fragment tensor-core bypass and sparse-row projection paths, using CUDA, CUTLASS, FlashAttention and PyTorch components. K049 is its searched predecessor; K050 adds fusion of the two input LayerNorms while preserving their separate affine parameters and thresholds. The all-skips-off control retains fusion while disabling both attention and projection skips; the attention-dense control retains projection skips. The historical ablation summary forms ratios from separately measured, native-normalized timings. The matched gains in Appendix D.5 instead directly divide candidate latency geometric means, as defined below. K050's sparse paths improve 14 of 30 checkpoints relative to all-skips-off, with geometric mean ratio $1.0432\times$. Disabling attention skipping improves all 30: at the search checkpoint, instrumentation confirms that skipping bypasses 58.0% of eligible QK and 67.2% of PV tensor-core matrix multiply-accumulate (MMA) instructions, yet its overhead exceeds the saved time. Scalar zero-product fractions and skipped MMA fractions are different estimands.

The attention path tests whether either warp-distributed operand fragment contains only zeros, and issues an MMA only when both fragments contain a nonzero. Partial sparsity can therefore leave an entire instruction active. Zero detection, warp votes and branches recur inside the kernel; causal masking and softmax still execute. A zero QK score does not imply a zero softmax probability. The measurements do not identify a profitable joint-zero pattern or break-even threshold. They show that the skipped work does not repay these costs for the measured 14M shapes and hardware.

Development also tested shortcuts that reuse cumulative value sums for zero queries, detect non-overlapping nonzero positions in Q/K , or reduce the cost of conditional execution. Qualified versions did not establish a reliable attention-speed benefit; earlier aggressive alternatives failed numerical checks. These trials do not exhaust strategies that could avoid operand loading earlier. The frozen candidate is not an optimal latency configuration: its attention-dense control is faster.

Table 12: Numerically qualified full-model speedups against native SDPA CUDA graphs. GM is the equal-checkpoint geometric mean; the range is across qualified checkpoints, not a confidence interval. All conditions use one RTX5090, BF16, batch one, 2,048 tokens and full vocabulary logits. P0's four-checkpoint subset is shown only to document its limited qualification.

Implementation	Qualified	GM speedup	Range
K049	30/30	1.1141	0.9550–1.5284
K050	30/30	1.2340	1.0059–1.7832
K050, attention dense	30/30	1.2506	1.0185–1.8022
K050, all skips off	30/30	1.1829	1.0503–1.3628
P0 (qualified subset only)	4/30	0.8910	0.8180–1.0012

D.5 STRUCTURED SPARSITY AND MATCHED RUNTIME DIAGNOSTICS

This subsection preserves the original 30-checkpoint diagnostic subset; the five restored h-only records and updated 35-point fit are reported below.

For the original diagnostic subset, we reconcile the same 30 checkpoints with their canonical logical-product counts, retained kernel diagnostics, and raw matched timing pairs. Table 13 separates native and optimized latencies for each checkpoint. Let t_N, t_0, t_P, t_K denote the geometric-mean latencies of native execution, all-skips-off K050, projection-only K050, and full K050. The gains from enabling projection and attention skipping are t_0/t_P and t_P/t_K , respectively. Both controls retain K050’s fusion; the former disables attention skipping in both implementations. These are full-model latency ratios, not timings of the projection operations alone. t_N/t_0 compares native execution with the optimized all-skips-off control, which also changes dense kernels and layouts, so it does not isolate fusion. Native-relative full speedup is t_N/t_K ; t_N uses the native measurements paired with full K050. Each geometric mean uses 64 inputs, seven paired passes, and three processes, matching the original timing protocol.

Execution units. QKV and FFN-up omit a $16 \times 8 \times 16$ MMA when the entire 16-token by 16-feature activation fragment is zero. Their counters are reconstructed from the actual BF16 operands using the implemented predicate. FFN-down and attention output process eight real token rows per padded 16-row MMA. Rows with at most two nonzeros may use scalar operations under the kernel’s numerical safety checks; remaining empty feature fragments bypass MMAs. These latter counters are instrumented and independently checked against operand-derived counts. Their bypass totals include replacement by scalar work as well as empty fragments, so they are not counts of eliminated arithmetic. We pool bypassed and eligible MMA counts across the four projection families before dividing, retaining the padding and scalar-work distinctions in the supplementary data.

Attention omits an MMA when either complete operand fragment is zero. In 29 checkpoints, QK skips 0% and PV skips 10.42% of eligible instructions; the PV baseline includes masked or padded probability work. Only seven-site + OL1(all) at $\kappa = 0.5$ has additional skipping, reaching 58.00% for QK and 67.21% for PV. Attention-skipping gain nevertheless remains below one for every checkpoint ($0.9832\text{--}0.9894\times$). These instruction counts include tiled causal padding, unlike the unmasked scalar logical-product denominator.

Associations and matched recipes. Figure 18 compares scalar and instruction-level predictors. Projection-skipping gain has Pearson $r = 0.973$, $R^2 = 0.946$ against projection MMA bypass, versus $r = 0.643$, $R^2 = 0.413$ against the projection scalar zero-product fraction. The within-family $\mathcal{S}_{\text{model}}$ versus native-relative speedup fits have $R^2 = 0.542, 0.784$, and 0.762 for the baseline/one-site, four-site, and seven-site groups (ten checkpoints each). Centering both variables within family gives $R^2 = 0.660$. The global relation is therefore not solely separation between family means; all fits remain descriptive and do not control for threshold, pressure, quality, or native cost. Attention MMA skipping has only two distinct predictor values, with one checkpoint at the larger value, limiting its correlation to that contrast.

Across all five matched OL1 thresholds, four-site recipes have greater projection scalar contribution, more projection MMA bypass, and larger projection-skipping gains than seven-site recipes. Without pressure, small bypass differences change sign, although four-site has the larger gain at each threshold. At $\kappa = 0.5$ with OL1, four-site bypasses 82.30% of projection MMAs versus 57.68% for seven-site. Its full K050 latency is also lower: 0.45948 versus 0.47337 ms. Seven-site nevertheless reports the larger native-relative speedup ($1.783\times$ versus $1.609\times$), because its native baseline is slower (0.84409 versus 0.73944 ms). This relative-speedup/absolute-latency reversal holds in all eight positive-threshold four-/seven-site pairs, with and without OL1.

The native implementation executes Q/K/V gates separately at nonzero thresholds; K050 fuses them with layout conversion and RoPE, including in the all-skips-off control. Similar all-skips-off latencies support native gate overhead as an explanation for the denominator difference, but do not isolate each operation’s timing contribution.

Coverage and unavailable quantities. Logical sparsity uses retained FP16 counts. Kernel counters use actual BF16 operands over all 338 validation blocks; timing uses the original 64-input subset. Pure-zero-only versus scalar-replacement bypass counts, counters restricted to the timed inputs, unmasked-only attention MMA counts, and isolated per-operation latency attribution are unavailable. No such values are inferred from $\mathcal{S}_{\text{model}}$. Supplementary `kernel/checkpoints.csv` retains all 30 records with operation counts, skip counters, latencies, and explicit missing fields;

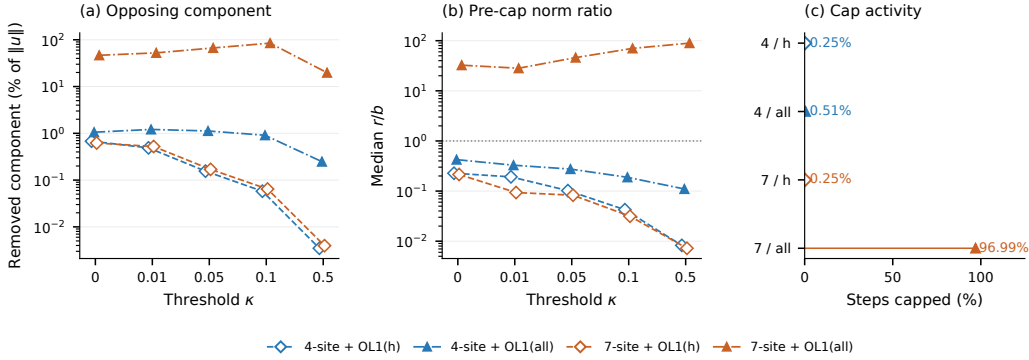


Figure 17: **H-only and broad pressure operate in different norm-budget regimes at 14M.** (a) Median removed opposing component, $100\rho_{\text{opp}}$; (b) median pre-cap ratio r/b at $\lambda = b = 1$. Each point summarizes 712 optimizer steps at a fixed threshold, not seed uncertainty. Color denotes threshold scope; diamonds/dashed lines denote h-only pressure and triangles/dash-dot lines denote broad pressure. (c) Cap-active fractions pool 3,560 steps per recipe and use the implemented condition $s < 1$. The geometry precedes group learning rates and decoupled weight decay, so it does not guarantee task-loss preservation. The h-only and broad-pressure comparison changes the objective composition and normalization.

kernel/associations.csv and kernel/matched-pairs.csv retain the complete fits and matched contrasts.

The five restored four-site h-only records are given in Table 14. Adding them to the original 30 changes the projection-bypass R^2 from 0.946 to 0.941, and the scalar-projection R^2 from 0.413 to 0.425. These expanded associations use all 35 qualified 14M records; historical association panels retain their explicitly stated 30-point subset. Attention skipping remains slower than attention-dense execution for all 35 historical comparisons. Seven-site h-only full-model timing is shown separately in Figure 1; no corresponding skipping ablations are added here.

Table 2 reports a separate single-checkpoint experiment at 14M T_7/P_{all} , $\kappa = 0.5$, disabling each operation’s skipping path while retaining weights, gates and the h/z fusion. The all-path control and untouched K050 implementation average 0.521 and 0.518 ms, respectively, with overlapping ranges across three processes. Each table span runs from the smallest path-off process mean minus the largest all-path mean to the largest path-off mean minus the smallest all-path mean. These descriptive spans are not confidence intervals. The ablations measure net path effects, without separately timing memory reads, zero checks or softmax.

D.6 PRESSURE-BUDGET DIAGNOSTICS AT 14M

Pressure scope also changes the realized optimization regime (Figure 17). All 20 pressured multisite 14M runs retain 712 optimizer-step records. We measure the task-relative opposing component $\rho_{\text{opp}} = \max(0, -\langle u, w \rangle) / (\|u\|_2^2 + \epsilon)$ in the global AdamW-preconditioned geometry. H-only pressure is capped on 0.25% of steps under either threshold scope; broad four-site pressure is capped on 0.51%, versus 96.99% for broad seven-site pressure. The pooled median pre-cap ratio r/b is 0.102 and 0.086 for four-/seven-site h-only pressure, compared with 0.253 and 57.49 for broad pressure. These logs establish different magnitude and budget regimes; they do not identify the cause of the validation-loss differences.

D.7 EVIDENCE AND DATA ACCOMPANYING THE DRAFT

The supplementary data contain the original 54 training endpoints plus ten verified 14M h-only endpoints, all 540 historical post-hoc evaluations, seven signed histograms, and the original 30-checkpoint kernel reduction. The pressure-placement extension adds both loss passes, site/layer counters, OL1 geometry, matched contrasts and five historical h-only runtime records. The Figure 1

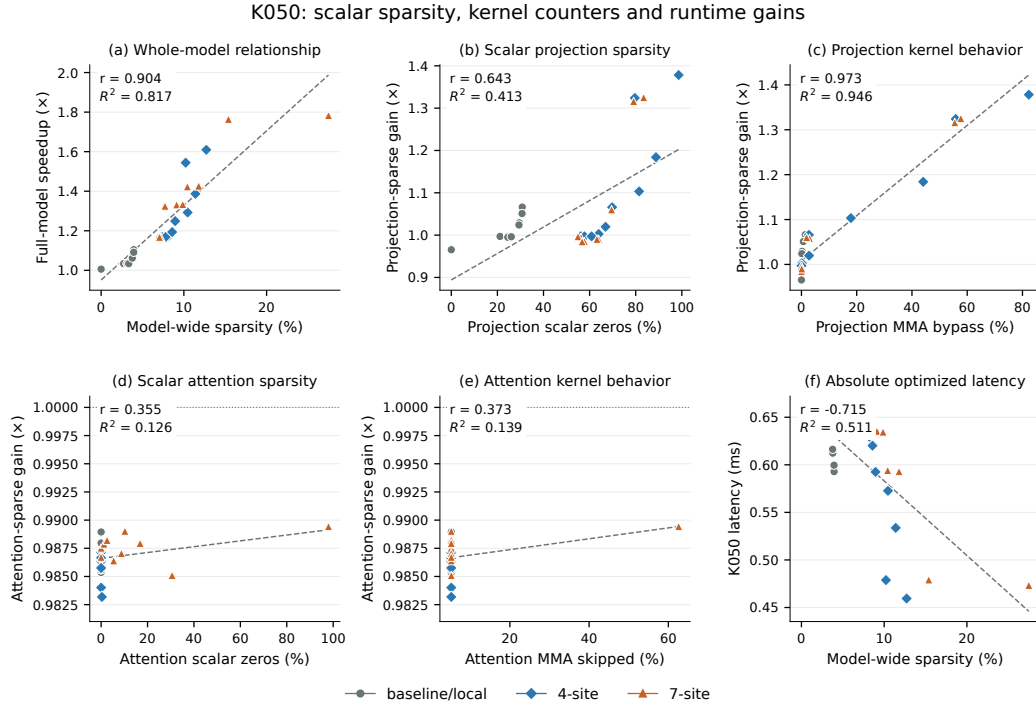
Table 13: **Matched K050 latency ablations for the original 30-checkpoint subset.** Entries are geometric-mean full-model latencies in milliseconds. Native uses the reference paired with full K050. All-skips-off retains the optimized implementation but disables both sparse paths; projection-only enables projection skipping with attention skipping disabled. The dose is λ for local L1/OL1 and κ for four-/seven-site recipes; multisite OL1 fixes $\lambda = 1$. Ratios should be computed from the unrounded supplementary values. Differences across checkpoints are not independent-seed estimates.

ID	Recipe	λ/κ	Native	All skips off	Proj. only	Full K050
c01	A0	–	0.65544	0.62147	0.64373	0.65157
c02	A1-H	–	0.65417	0.62230	0.62420	0.63282
c03	A1-H-L1	0.05	0.65435	0.62190	0.62375	0.63302
c04	A1-H-L1	0.1	0.65422	0.62238	0.62408	0.63251
c05	A1-H-L1	0.5	0.65447	0.62209	0.60448	0.61226
c06	A1-H-L1	1	0.65455	0.62296	0.58428	0.59282
c07	A1-H-OL1	0.05	0.65413	0.62234	0.62557	0.63256
c08	A1-H-OL1	0.1	0.65446	0.62224	0.62467	0.63308
c09	A1-H-OL1	0.5	0.65438	0.62290	0.60831	0.61633
c10	A1-H-OL1	1	0.65426	0.62181	0.59167	0.59960
c11	A4	0	0.73993	0.62327	0.62495	0.63391
c12	A4	0.01	0.74030	0.62344	0.62535	0.63391
c13	A4	0.05	0.74012	0.62313	0.62148	0.62958
c14	A4	0.1	0.73999	0.62294	0.58446	0.59255
c15	A4	0.5	0.73940	0.62389	0.47116	0.47882
c16	A4-OL1(all)	0	0.74048	0.62294	0.62479	0.63328
c17	A4-OL1(all)	0.01	0.74015	0.62394	0.61189	0.62014
c18	A4-OL1(all)	0.05	0.74025	0.62340	0.56503	0.57268
c19	A4-OL1(all)	0.1	0.74017	0.62296	0.52610	0.53371
c20	A4-OL1(all)	0.5	0.73944	0.62263	0.45175	0.45948
c21	A7	0	0.74015	0.62266	0.62605	0.63396
c22	A7	0.01	0.84616	0.62117	0.63061	0.63835
c23	A7	0.05	0.84608	0.62064	0.62669	0.63533
c24	A7	0.1	0.84553	0.62065	0.58647	0.59417
c25	A7	0.5	0.84523	0.62121	0.47200	0.47914
c26	A7-OL1(all)	0	0.74032	0.62343	0.62561	0.63403
c27	A7-OL1(all)	0.01	0.84579	0.62111	0.63089	0.63842
c28	A7-OL1(all)	0.05	0.84593	0.62147	0.62755	0.63452
c29	A7-OL1(all)	0.1	0.84583	0.62105	0.58583	0.59298
c30	A7-OL1(all)	0.5	0.84409	0.62071	0.46837	0.47337

Table 14: Restored four-site h-only runtime measurements. All entries use full-model geometric-mean latency in ms, pooled from the original matched timing pairs. These five records extend the original 30-checkpoint analysis without new execution.

κ	Loss	Native	All skips off	Projection only	Full K050
0	5.2157	0.74038	0.62311	0.59625	0.60366
0.01	5.2045	0.74027	0.62376	0.58440	0.59216
0.05	5.1956	0.74012	0.62376	0.55392	0.56180
0.1	5.2287	0.74001	0.62433	0.52171	0.52931
0.5	5.7227	0.73957	0.62339	0.45587	0.46225

evidence export additionally records all 44 plotted trained checkpoints, including the ten 70M h-only endpoints, and the 40 control-clipping settings with their matched latency, checkpoint identities, timing sessions and source hashes. The Figure 4 export retains the same 84 settings, their unrounded latencies and the two common base-model references used to compute every displayed speedup. The Figures 3 and 14 show 70 of the original 74 trained endpoints and all 60 base/ReLU clipping settings. The four historical naive-L1 endpoints are retained in the export but omitted from the main figure. The split export records plotted and excluded membership, display limits, uniform final-loss convention and pooled counts; the original retains the protocol checks. The Figure 5 export retains all 20 all-site-minus-h-only pressure contrasts, including the two small 14M latency reversals at $\kappa = 0.5$; its checkpoint identities match the Figure 1 endpoint export. The same derived-data file retains twelve operation decompositions, with per-operation integer numerators, the common full-model denominator and checkpoint identities. The Table 2 export retains Run037's single-checkpoint operation ablations, issued and bypassed instruction counts, scalar replacement counts, unrounded conditional effects, process spans and control timings with source identities. These measurements form a separate timing session from the historical figures. The earlier four-checkpoint 14M/70M bypass figure and its complete export remain available in the generating analysis. The README defines coverage and units; `SOURCES.json` records copy hashes and `protocol.json` records implementation and data identities. These files accompany the repository, not a standalone checkpoint or kernel release.



30 checkpoints. Counters: 338-block BF16 diagnostic; timings: 64-input subset. MMA bypass includes SIMT substitution and padding.

Figure 18: Scalar sparsity, skipped instructions, and matched runtime gains. The 30 checkpoints share colors and shapes by family. Projection and attention gains are t_0/t_P and t_P/t_K ; lines are descriptive OLS fits with intercepts. Projection bypass includes scalar substitution and padded MMA rows. Attention skip fractions include masked/padded work and take only two distinct values. The remaining pages of the accompanying diagnostic PDF show within-family fits, matched threshold comparisons, and absolute latency traces. Counters cover 338 validation blocks, while timings cover 64 matched inputs.